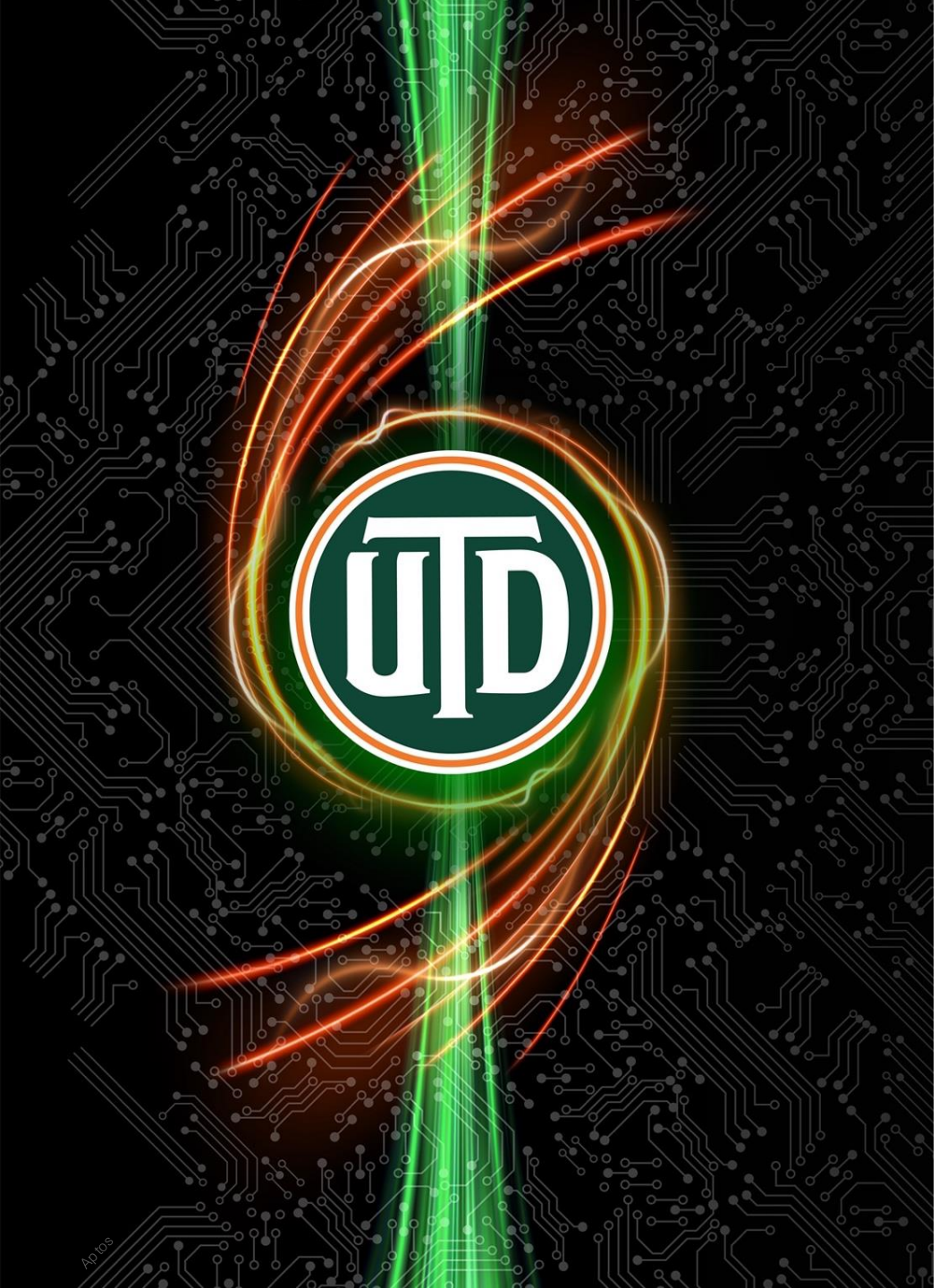
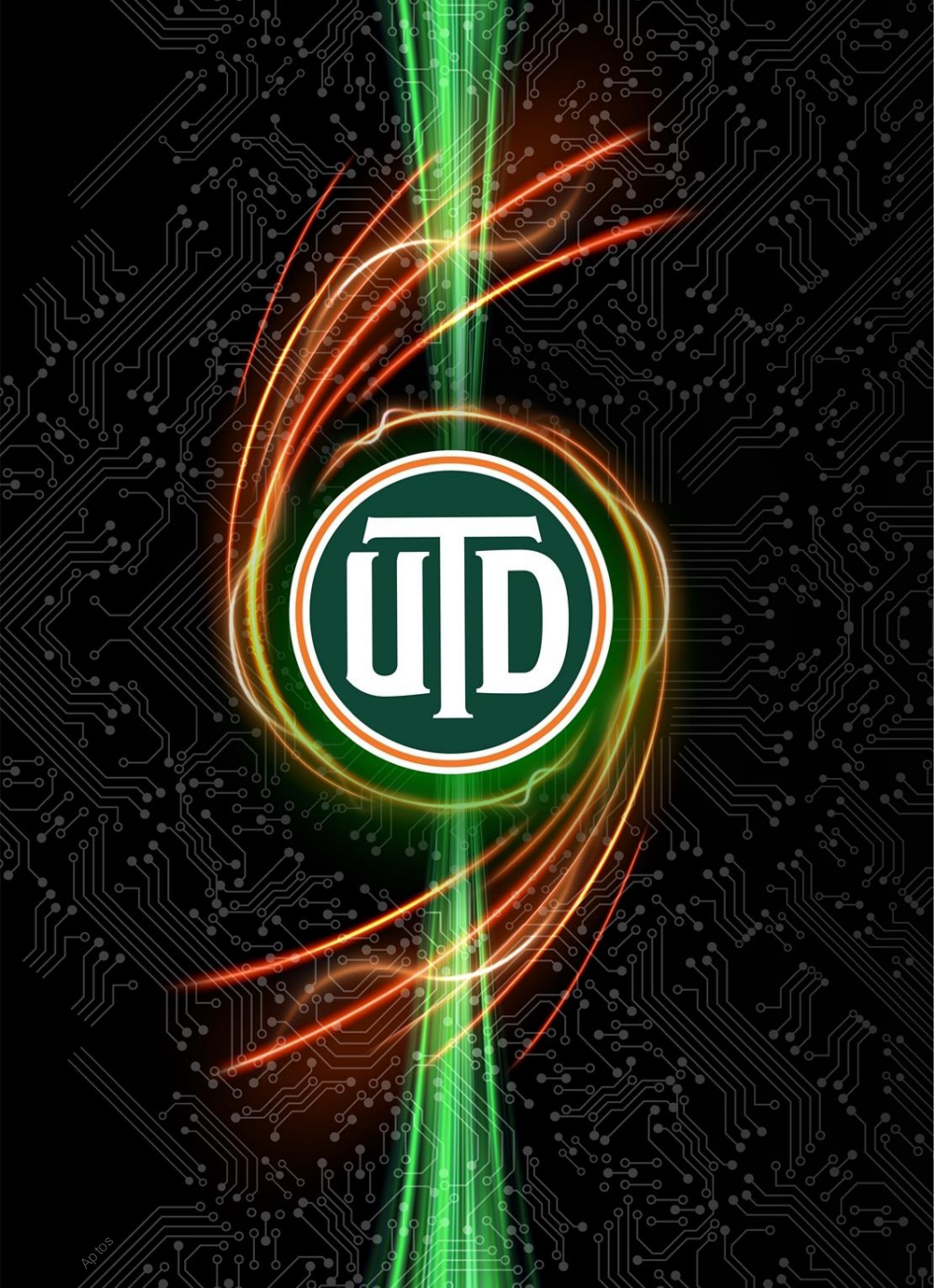


The latest version of this presentation is available at https://hpc.utdallas.edu/systems-resources/juno/.
Please always use the latest version for reference.

# Juno Orientation

Fall 2025 v11, October 2025

HPC@UTD



# Juno Orientation

Fall 2025 v12, October 2025

HPC@UTD

# Outline

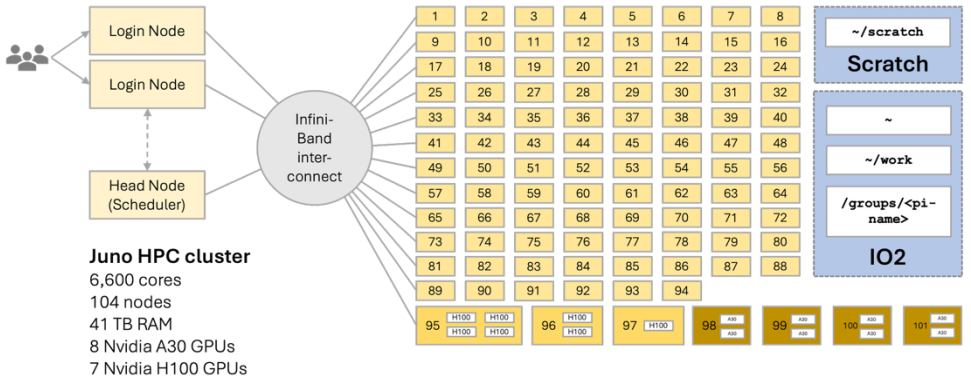
- Juno is an HPC cluster
- Directories and data management
- Basic Unix commands summary
- Logging into Juno and transferring data
- Introduction to the SLURM job scheduler
- Achieving high performance using parallel processing
- Getting help



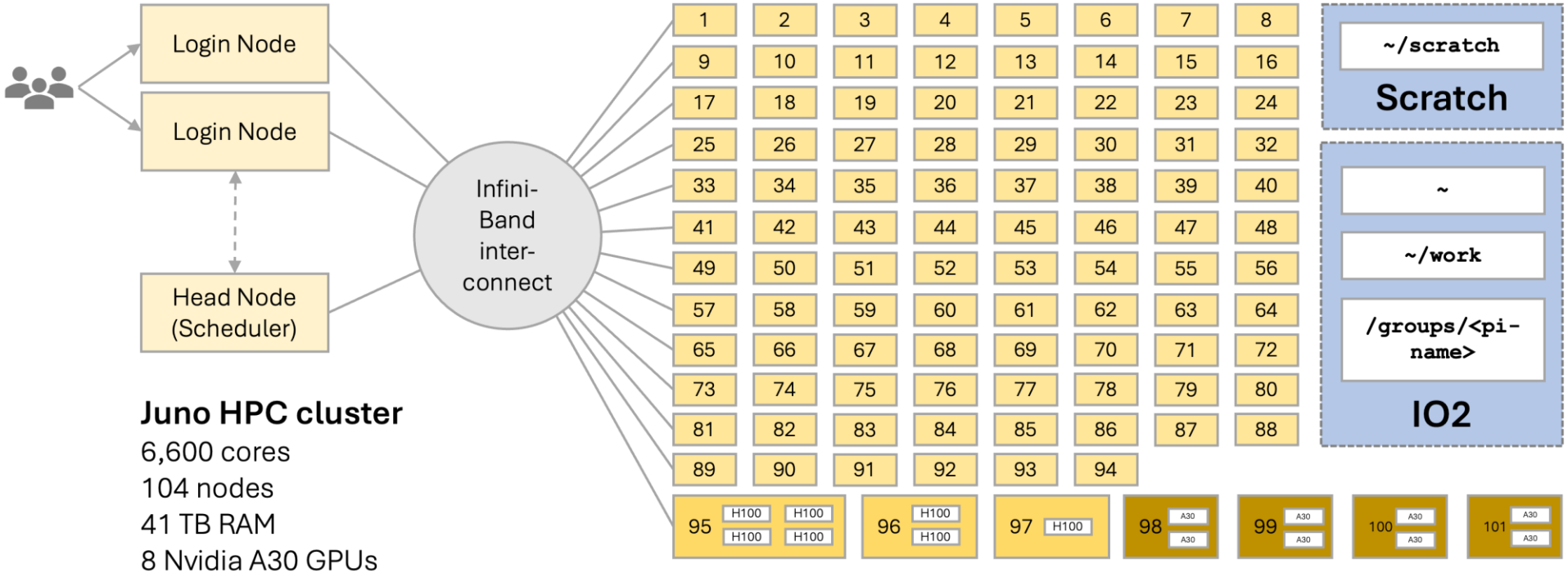
# Juno is an HPC cluster

# Juno is an HPC cluster

- An HPC cluster consists of several computers that work together to run programs very fast.
  - Each individual computer is called a node.
- HPC cluster nodes have different roles. These are:
  - Compute nodes: CPU nodes, GPU nodes
  - Login nodes
  - Head (or scheduler) nodes
- HPC clusters also have access to high performance storage systems
  - Storage holds your data and programs
- Nodes are interconnected using high speed networks

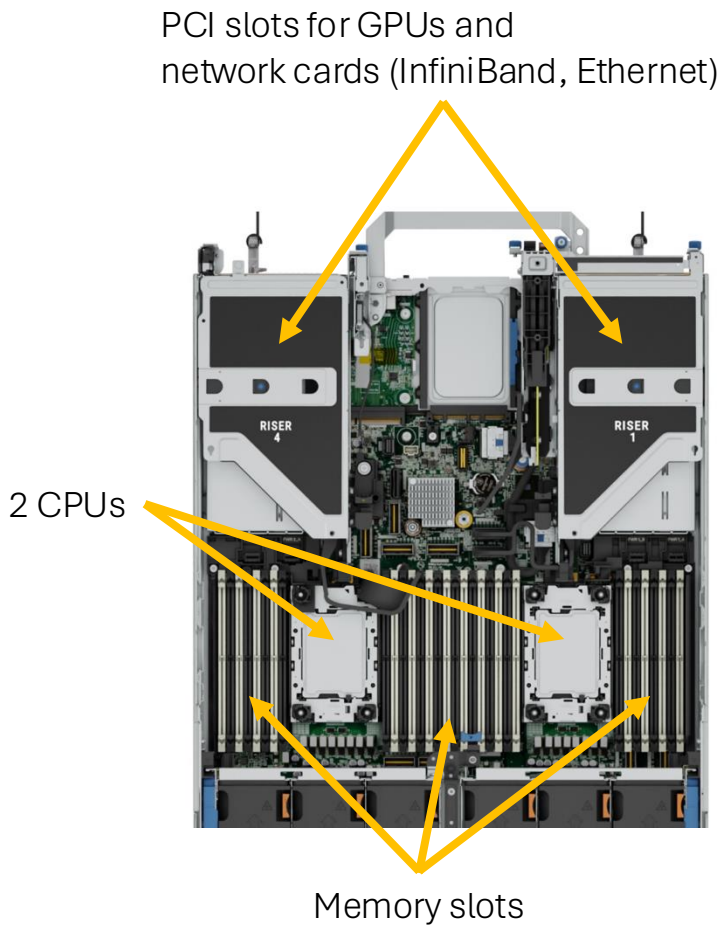


# Juno’s configuration

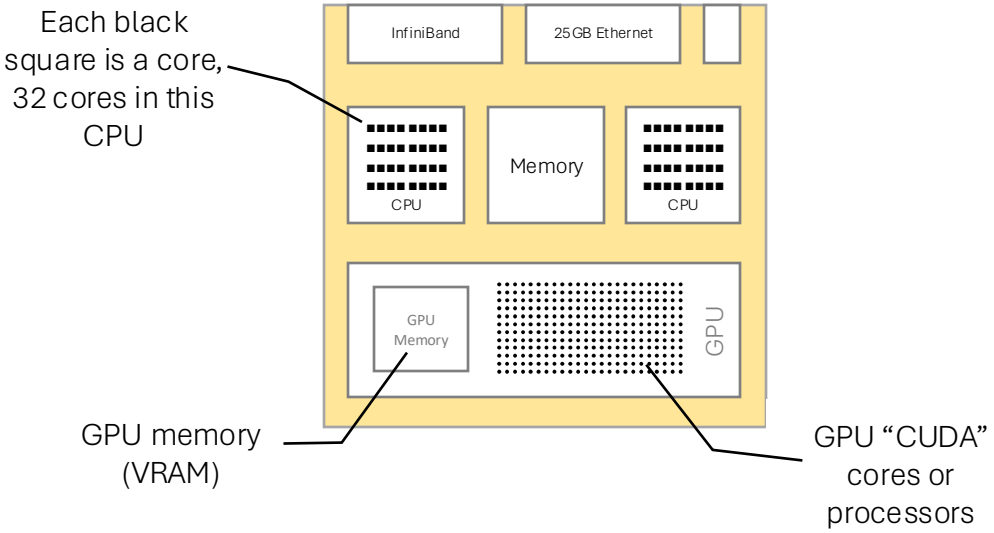



**Juno HPC cluster**
6,600 cores
104 nodes
41 TB RAM
8 Nvidia A30 GPUs
7 Nvidia H100 GPUs

# Terminology: CPUs, cores, VRAM, GPU cores

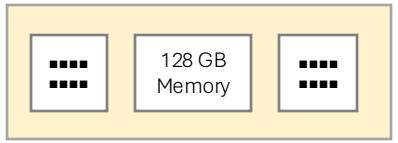



- CPU refers to a chip that installs in a motherboard socket
  - A CPU contains several cores
- Core refers to a processor capable of running a program

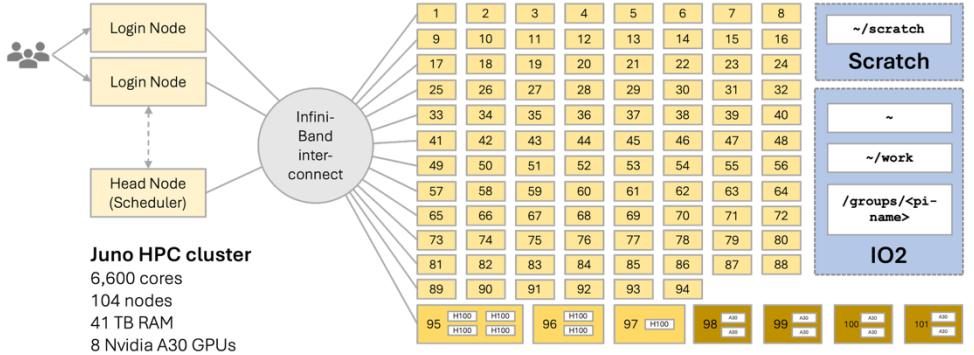



Source: https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/poweredge-r760xa-technical-guide.pdf

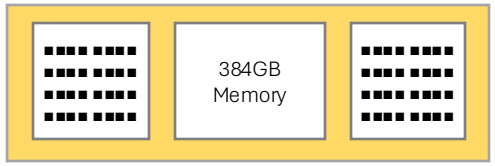
# Juno’s node configuration



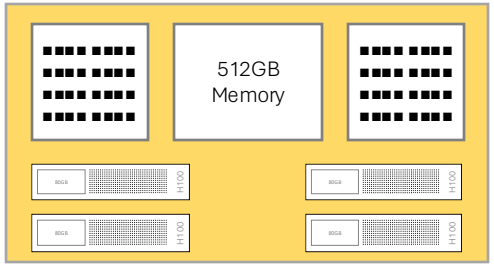

**Login and Head Nodes (3)**
2x Intel Xeon Sliver 4309Y 2.8GHz
8C/16T 12M cache
128 or 384 GB RAM (8x16GB),
3.84TB (2x1.92TB SSD)
HDR100 InfiniBand, 2x1Gbps
Ethernet



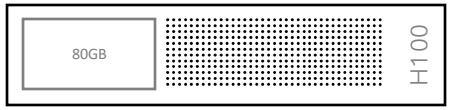

**Juno HPC cluster**
6,600 cores
104 nodes
41 TB RAM
8 Nvidia A30 GPUs
7 Nvidia H100 GPUs

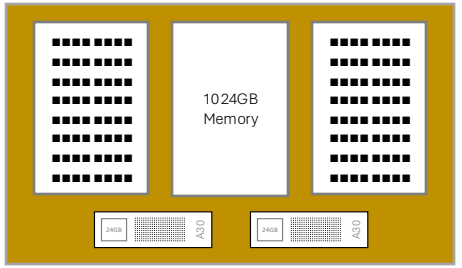



**Compute Nodes (94)**
2x AMD EPYC 9334 2.7GHz 32C/64T 128M
cache
384GB RAM (12x32GB), 480GB SSD
HDR100 InfiniBand, 10/25Gbps Ethernet

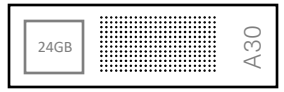



**GPU Compute Nodes (1x4H100, 1x2H100, 1xH100)**
2x Intel Xeon Platinum 8462Y 2.8GHz 32C/64T 60M cache
4x Nvidia HGX H100 80GB HBM3 GPUs, 2x Nvidia H100 94GB
NVL GPUs, 1x Nvidia H100 94GB NVL GPU
512 GB RAM (32x16GB), 480GB
HDR100 InfiniBand, 2x10/25Gbps Ethernet, 2x1Gbps Ethernet



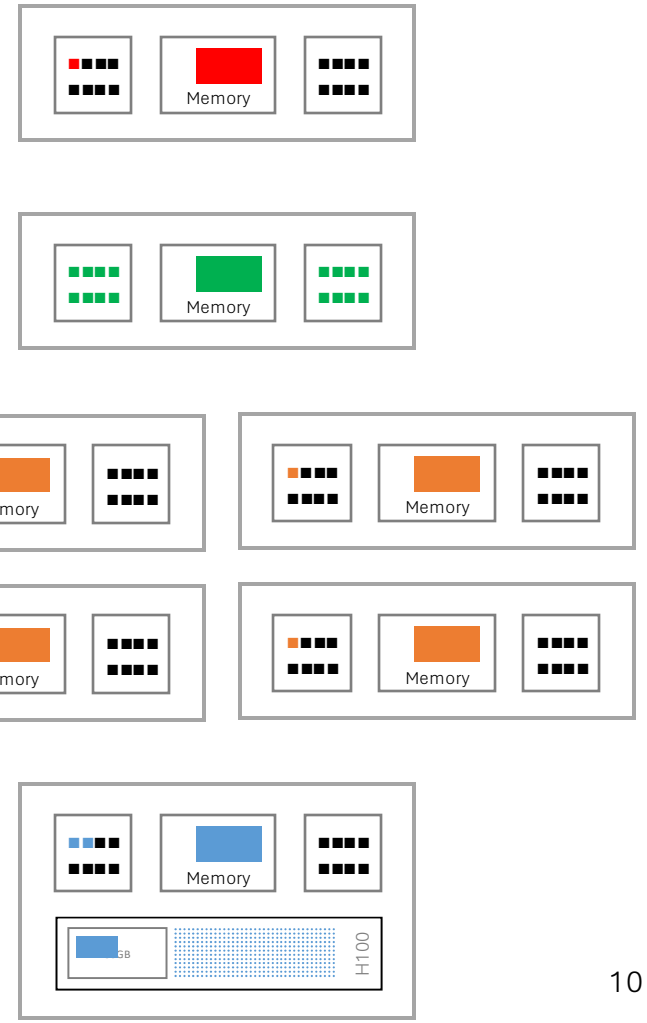




**GPU Compute Nodes (4x2A30)**
2x AMD EPYC 9534 2.45GHz 64C/128T 256M cache
2x Nvidia A30 24GB GPUs
1024GB RAM (16x64GB), 480GB SSD
HDR100 InfiniBand, 2x10/25Gbps Ethernet, 2x1Gbps
Ethernet



# Achieving high performance on Juno

- Programs normally perform their work using just one core
  - This is called serial execution
- Achieving high performance requires the program to split its work and perform it simultaneously on multiple cores
  - This is called **parallel execution**
- HPC cluster support 3 modes of parallel execution
  - Shared-memory multiprocessing parallel (aka, multithreaded parallel) mode uses multiple cores in one node
    - Program uses OpenMP for parallelization
  - Message passing parallel (aka, distributed memory parallel) uses multiple nodes
    - Program uses the MPI library for parallelization
  - GPU acceleration uses tiny CUDA cores for parallelism
    - Program uses CUDA or OpenMP for GPUs for GPU acceleration



# Software available on Juno

- Compilers and toolkits:
  - GNU, Intel
  - OpenMPI, MPICH
  - CUDA
  - AMD μProf, Valgrind
  - Launcher
  - Spack
  - Apptainer (container)
  - Miniconda
- Software and packages:
  - MATLAB
  - Gaussian
  - Ansys (Fluent, Lumerical,…)
  - GROMACS, AMBER, NAMD
  - R, Python
  - ORCA, QChem, VASP
  - Stata
  - Ollama

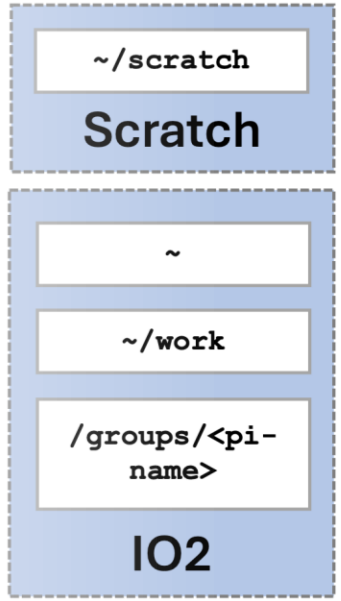
  and more…



# Directories and data management

# HPC storage systems

- **IO2** is the high-speed storage system for storing programs and data used actively computations. Your IO2 directories are
  - Home `~`
  - Work `~/work`
  - Group `/groups/<PI-name>`
- **Scratch** is the very high-performance storage system for high-speed IO during batch jobs
  - Scratch `~/scratch`



# Data directories

| Home | Work | Group |
| --- | --- | --- |
| `~` | `~/work` | `/groups/<pi-name>` |
| 50GB<br>Private to user, backed up daily | 1TB<br>Private to user, backed up daily | 1TB or more<br>Shared by the research group, backed up daily |
| Use for configuration files, login scripts, user installed software, small input and output files | Use for user installed software, large software packages, large data files, results files | Use for group software and large datasets, models and results files |
| Do not use for batch jobs IO needs | Can use for batch jobs with light to moderate IO needs<br>• Use Scratch space for jobs requiring heavy IO | |

# Tracking your storage usage on IO2

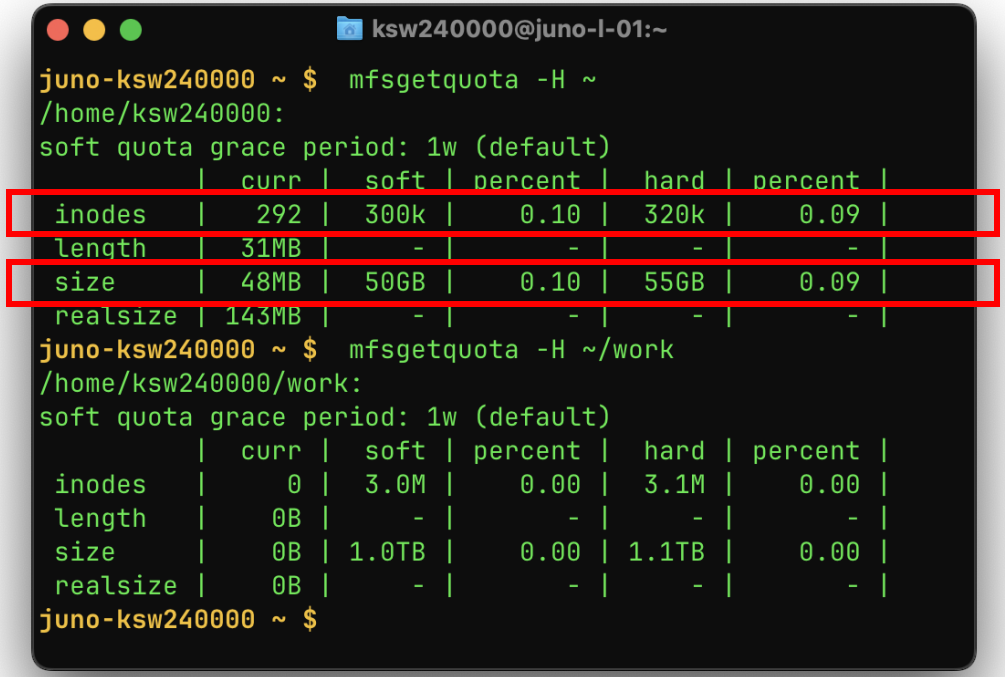
- Get your quota and usage using the command

  **`mfsgetquota -H <directory>`**

  e.g.,

  **`mfsgetquota -H ~/work`**

- Two kinds of quotas are implemented
  - Storage amount
  - Number of inodes (files and directories)



# Scratch space

## Scratch

**~/scratch**

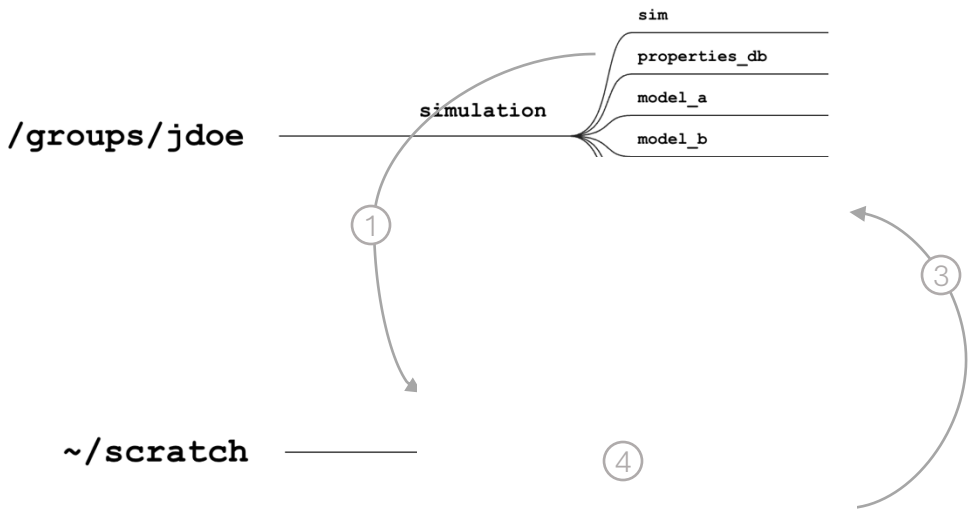
30TB (soft limit)

Private to user, never backed up, purged regularly

Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving



# Scratch space

## Scratch

**~/scratch**

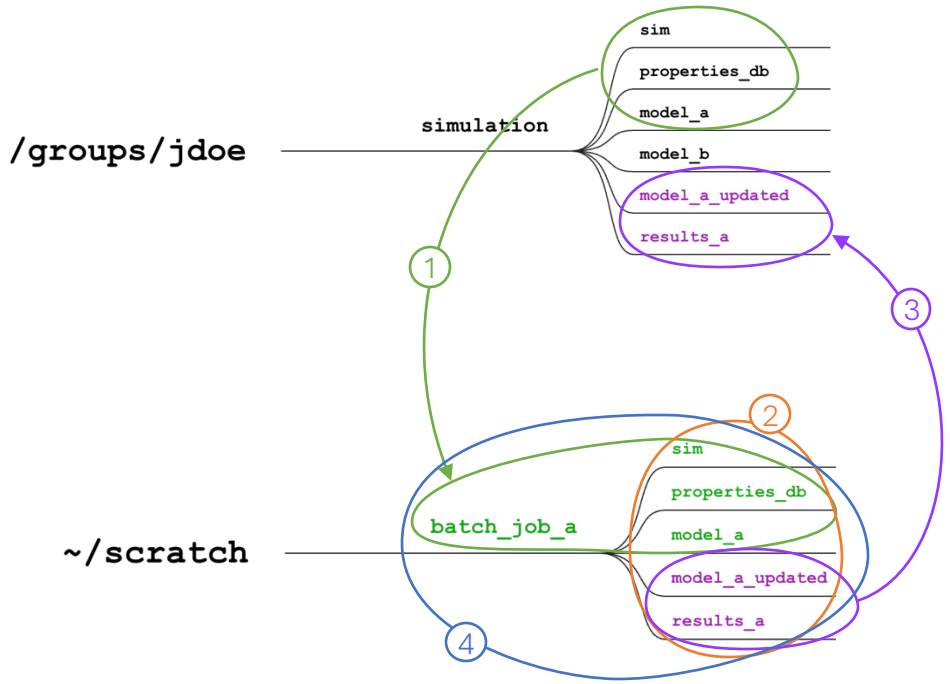
30TB (soft limit)

Private to user, never backed up, purged regularly

Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving



# Scratch storage management policy

- Overall Scratch space is not very large, so we need to keep it clean
  - I.e., use it during batch jobs, delete afterwards
  - We regularly purge (delete old files from) Scratch space
- Scratch storage management policy (https://hpc.utdallas.edu/policies-and-guidelines/) states

> HPC Scratch storage (**`~/scratch`** directory) is for use during a job's execution. Batch jobs should utilize Scratch as "*copy in, read and write during, copy out, and clean up when done*" data space.
>
> Data stored in Scratch is never backed up. Scratch will be purged regularly to maintain sufficient space for new batch jobs to run. When purged, any file or directory not accessed for 45 days will be deleted. The HPC staff will monitor Scratch utilization and, in the future, may reduce the (currently) 45 day time limit.



# Basic Unix commands



# Basic Unix commands (not covered here)

- Directories
  - `ls` – list contents of a directory
  - `pwd` – print/show current working directory
  - `cd` – change current directory
  - `mkdir` – make a new directory
  - `rmdir` – remove/delete a directory
- Files
  - `cp` – copy a file to a new name and/or location
  - `mv` – move a file to a new name and/or location
  - `rm` – remove/delete a file (or directory)
  - `grep` – search content of a file for some text
- Editor
  - `nano` – a full screen interactive editor to edit text files
- Remote login
  - `ssh` - establishes a secure connection from your computer to a remote computer
- Transferring data between systems
  - `scp` - x
  - `rsync` - x
- Bash shell
  - `bash` - the shell program that
    - accepts user commands, runs programs, and shows output on the screen
    - can run commands from a script file, run programs, and sends output to another file
  - Environment variables - store values that `bash` and other programs can use during execution
  - `PATH` - variable containing the set of directories bash searches to run a program
  - `LD_LIBRARY_PATH` - variable contains set of directories a program uses to link its dynamic libraries
- Other
  - `which` - shows the directory from which a given program will be executed
  - `ldd` - displays the directories from which each dynamically linked library will be loaded

# The `module` command

- Some software available on Juno (as of October 2025)…
  - GNU compilers versions 11, 12, 13, 14
  - Ansys, Fluent
  - Gaussian, Amber
- Each is installed in a different system directory

- How do I run version 12 of the GNU compilers?
  - Manually, set **`PATH`** and other environment variables for GNU compiler 12's install directory
  - Automatically using the **`module load gnu12`** command

- **`module`** manages PATH and other environment variables with simple commands (see https://modules.sourceforge.net/)
- **`module`** command can be used to find, list, load, and unload "modules" related to specific software and their versions

# The `module` command

```
$ module avail
$ module list
$ module load <module-name>
$ module unload [ <module-name> ]
```

- By default, the following modules are loaded by default on Juno (as of October 2025)

```
$ module list
Currently Loaded Modules:
  1) autotools    3) gnu14/14.2.0    5) ucx/1.17.0          7) openmpi5/5.0.7
  2) prun/2.2     4) hwloc/2.11.0    6) libfabric/1.18.0    8) ohpc
```



# Logging into Juno and transferring data

SSH (command line)

Open OnDemand (web based)

Transferring data into and out of Juno

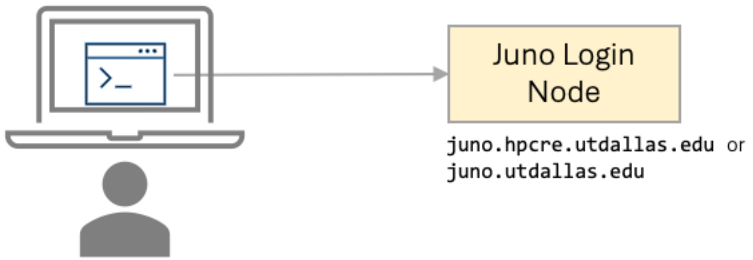
# Logging into Juno: ssh

- Your computer must be connected to the the UTD campus network via
  - Wired Ethernet connection on campus, or
  - WiFi connection to CometNet, or
  - A VPN connection to campus (see https://atlas.utdallas.edu/TDClient/30/Portal/Requests/ServiceDet?ID=167)
- Start a terminal program on your computer
- To login to Juno, run

  **`ssh <your-NetID>@juno.utdallas.edu`**
  e.g.,
  **`ssh ksw240000@juno.utdallas.edu`**
- **`ssh`** establishes a secure remote connection from your computer to the Juno login node

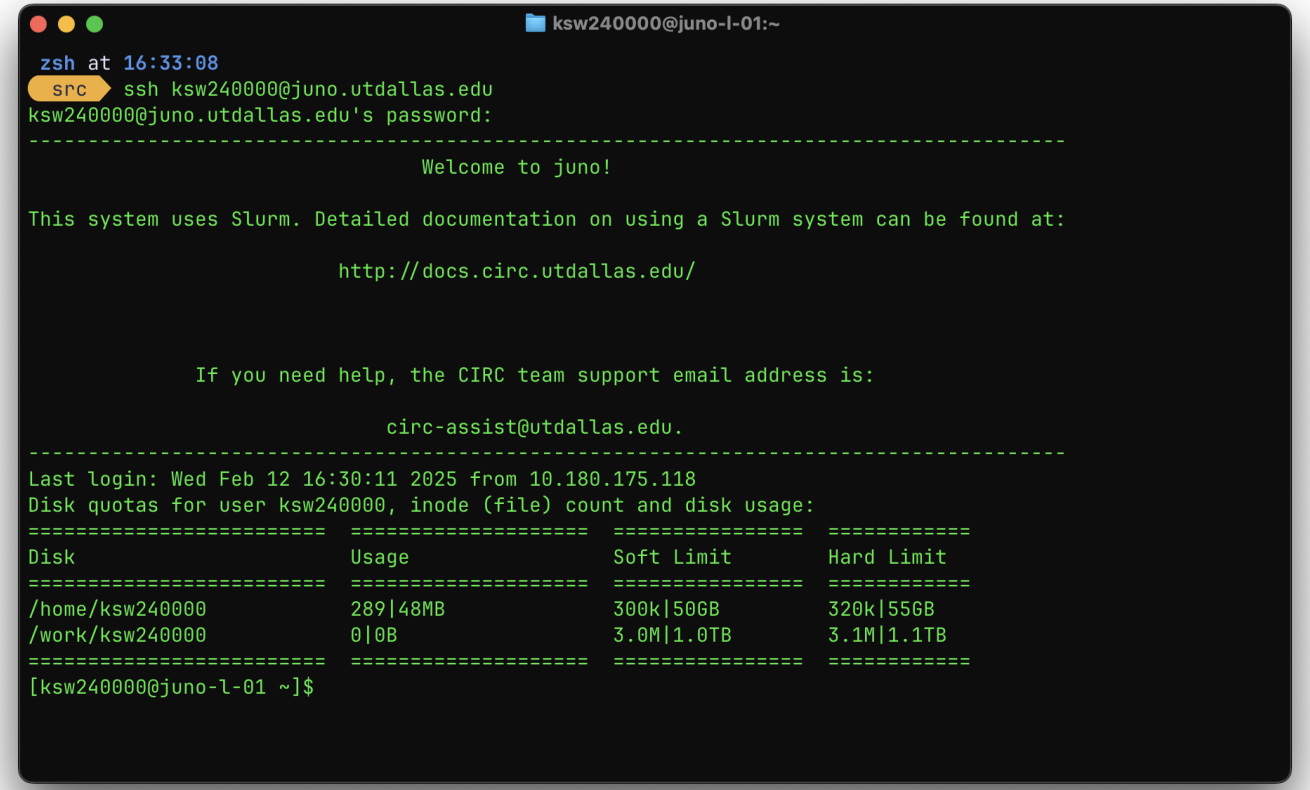
Recommended terminal programs

- Windows: Windows Terminal, MobaXTerm, or putty
- Mac: Terminal (pre-installed) or Ghostty
- Linux: terminal (pre-installed) or xterm (pre-installed)




Command line interface (CLI)

# Logging into Juno: ssh



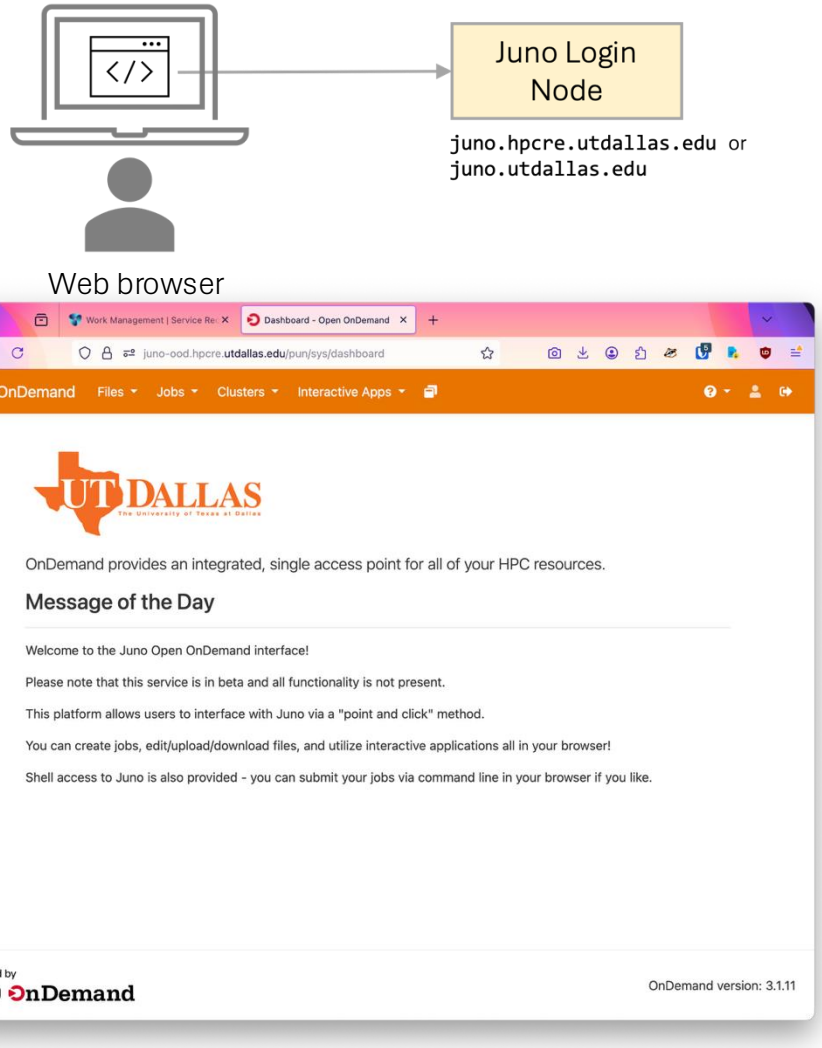

- You will be asked to
  - Verify that you want to connect to Juno (once only, not show on left)
  - Enter your password (associated with your NetID)
- You will see
  - A welcome message
  - Current disk quotas
  - Juno command prompt

```
[ksw240000@juno-l-01 ~] $
```

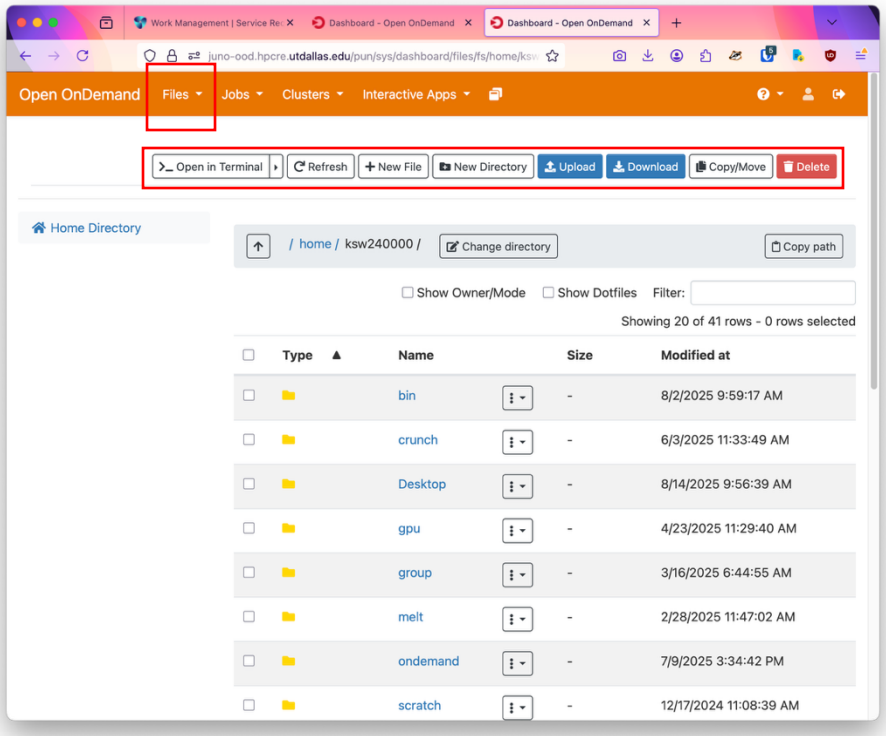
- You are ready to use Juno

# Logging into Juno: Open OnDemand

- Juno Open OnDemand portal lets users transfer files and run select programs
  - https://juno-ood.hpcre.utdallas.edu/
- Login using NetID and password



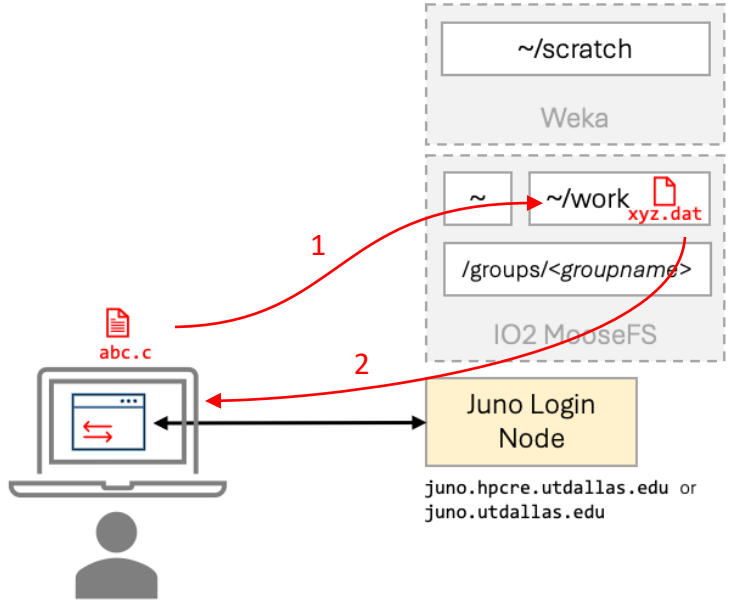
# Open OnDemand files access




- You can browse files
- Upload files
- Download files

# Transferring data

- You can transfer data between your computer and Juno using a data transfer program
- Examples of data transfer programs
  - **`scp`**, **`rsync`**
  - Filezilla




```
1 $ scp abc.c ksw240000@juno.utdallas.edu:~/work
2 $ scp ksw240000@juno.utdallas.edu:/work/xyz.dat xyz.dat
```



# rsync

- **`rsync`** can be useful to run on slow and unreliable connections.
- If your download aborts in the middle of a large file, **`rsync`** will be able to continue from where it left off when invoked again.

```
rsync -P <local-file> <remote-user>@<remote-system>:<remote-directory>
```

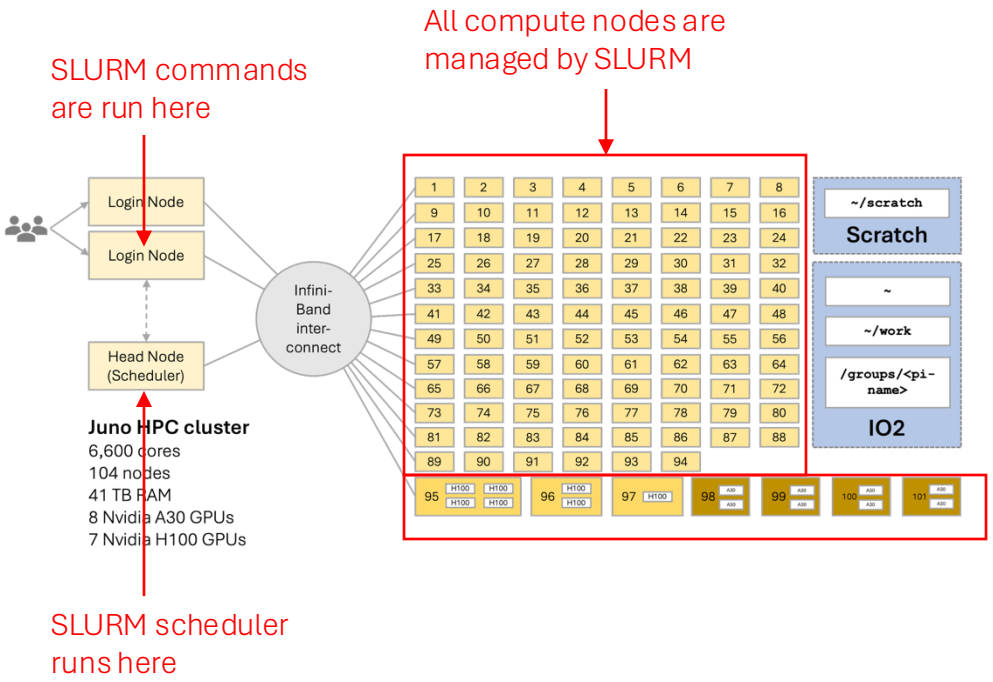
- The **`-P`** option preserves partially downloaded files and also shows progress.



# Introduction to the SLURM job scheduler

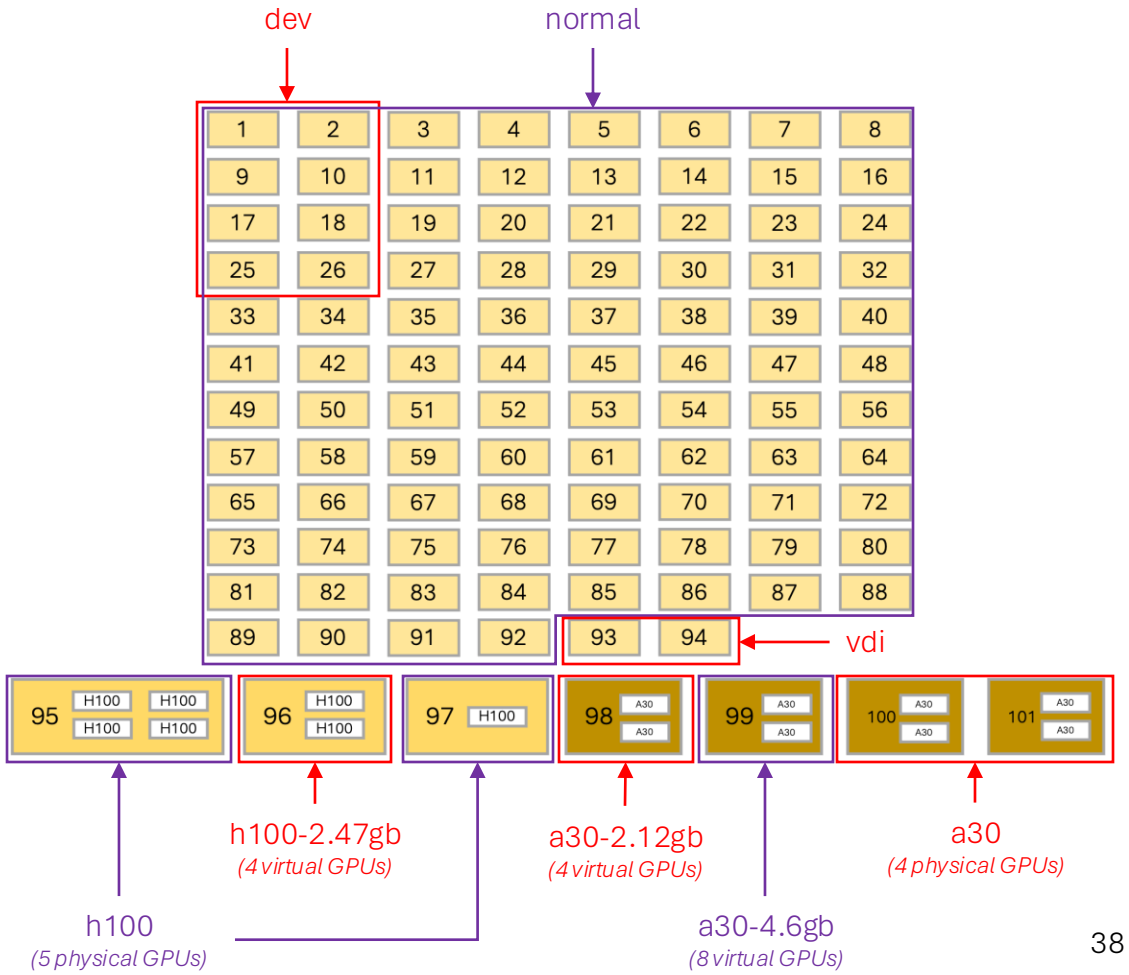
# What is the SLURM job scheduler

- SLURM is a job scheduler that manages the programs running on the compute nodes.



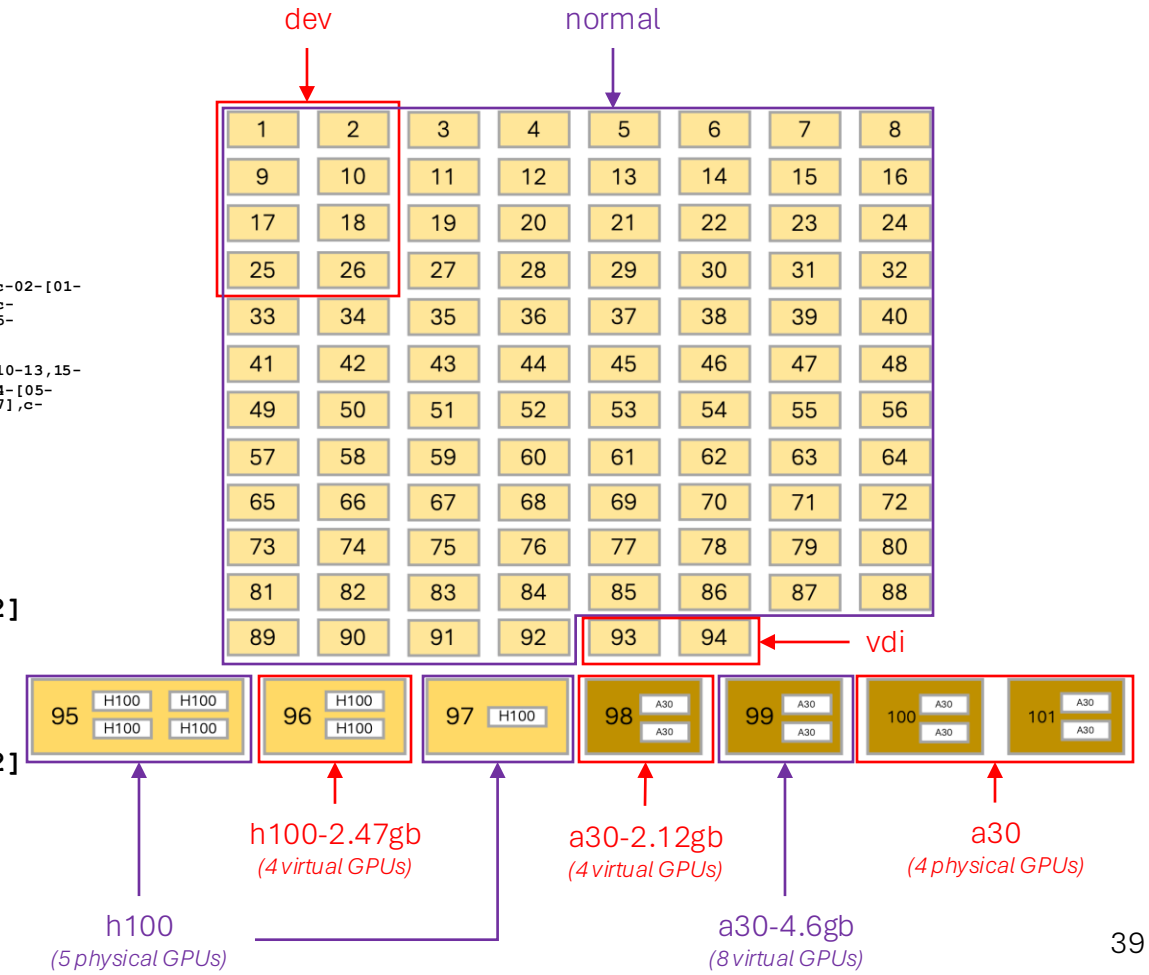
# SLURM partitions (often also called queues)

- Juno nodes are grouped into partitions. Each partition may have the following resources:
  - Nodes
  - CPUs (cores)
  - Memory
  - GPUs (full physical or fractional "virtual")
  - Time limit (how long a job can run)
- You submit your jobs to the partition that has the resources matching your job's needs
- SLURM runs the job when the resources become available
- **`sinfo`** command shows the currently configured SLURM partitions



# SLURM partitions (sometimes called queues)

```
[ksw240000@juno-l-01 ~]$ sinfo
PARTITION     AVAIL  TIMELIMIT  NODES  STATE NODELIST
normal*          up 2-00:00:00      1  drain c-01-03
normal*          up 2-00:00:00      1    mix c-01-12
normal*          up 2-00:00:00     35  alloc c-01-[04-11,17-18],c-02-[01-09,14],c-03-[17-18],c-04-[01-04,11-14],c-05-[08-09],c-06-[05-07]
normal*          up 2-00:00:00     48   idle c-01-[13-16],c-02-[10-13,15-18],c-03-[01-16],c-04-[05-10,15-18],c-05-[01-07],c-06-[09-11]
a30              up 2-00:00:00      1  alloc g-01-01
a30              up 2-00:00:00      1   idle g-02-01
a30-2.12gb       up 2-00:00:00      1   idle g-03-01
a30-4.6gb        up 2-00:00:00      1   idle g-04-01
dev              up    2:00:00      2   idle c-01-[01-02]
h100             up 2-00:00:00      1    mix g-05-01
h100             up 2-00:00:00      1   idle g-04-02
h100-2.47gb      up 2-00:00:00      1   idle g-06-01
vdi              up    8:00:00      2   idle v-05-[01-02]
```

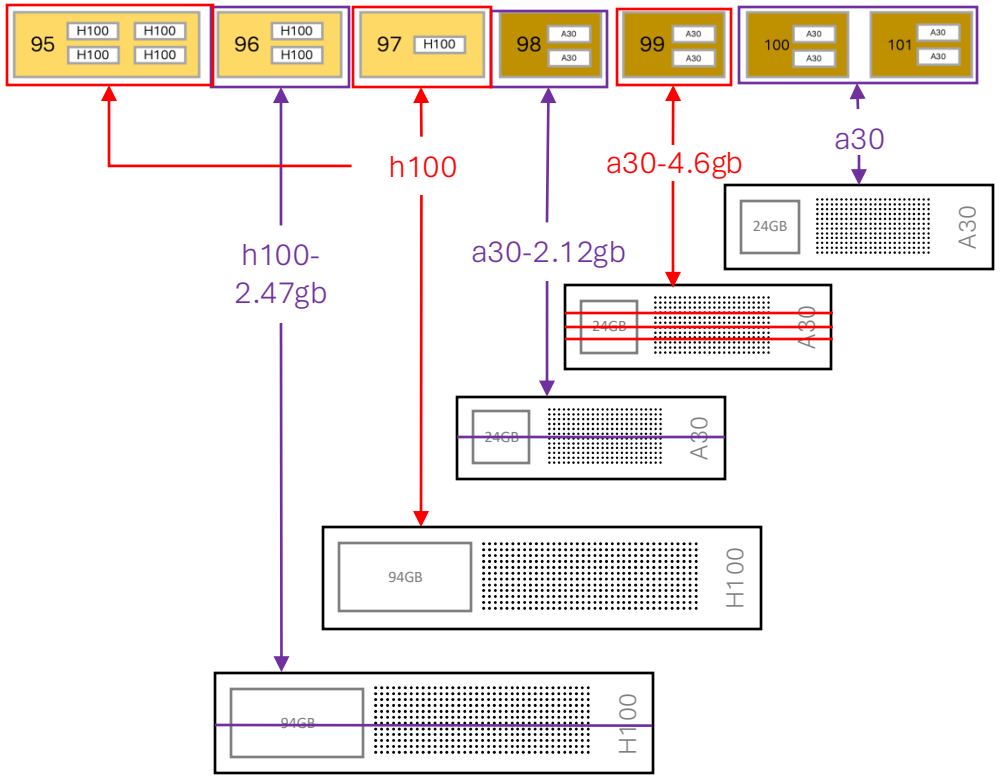


# What are these a30 and h100 partitions?

- Juno has 7 H100 and 8 A30 GPUs physical
- A GPUs can be “sliced” into multiple smaller “virtual” GPUs
- The available physical and virtual GPUs are

| | |
|---|---|
| **a30** | 4 physical GPUs (24GB each) |
| **a30-2.12gb** | 4 virtual GPUs (12GB each) |
| **a30-4.6gb** | 8 virtual GPUs (6GB each) |
| **h100** | 1 physical GPUs (94GB each) |
| **h100** | 4 physical GPUs (80GB each) |
| **h100-2.47gb** | 4 virtual GPUs (47GB each) |
| Total | 25 physical and virtual GPUs |

- You submit your job to the **a30*** or **h100*** partition corresponding to the type of GPU you wish to use



# Juno SLURM configuration

| Partition name | Time limit | Nodes | Max nodes/job | Cores/ node | Memory/ node | GPUs/ node | VRAM/ GPU | Best used for |
|---|---|---|---|---|---|---|---|---|
| **dev** | 2 hours | 8* | 8 | 64 | 384 GB | - | - | Code development, short jobs, benchmarking |
| **normal** | 2 days | 92* | 4 (default) or 8# (by request for high performance codes) | 64 | 384 GB | - | - | Long jobs, big jobs, production (main) compute workloads |
| **h100** | 2 days | 1 | 1 | 64 | 512 GB | **4** H100 physical | 80 GB | Large, long jobs requiring high GPU resources |
| | | 1 | 1 | 64 | 512 GB | **1** H100 physical | 94 GB | |
| **h100-2.47gb** | 2 days | 1 | 1 | 64 | 512 GB | **4** half-H100 virtual | 47 GB | |
| **a30** | 2 days | 2 | 2 | 128 | 1,024 GB | **2** A30 physical | 24 GB | Large, long jobs requiring medium GPU resources |
| **a30-2.12gb** | 2 days | 1 | 1 | 128 | 1,024 GB | **4** half-A30 virtual | 12 GB | Large, long jobs requiring small GPU resources |
| **a30-4.6gb** | 2 days | 1 | 1 | 128 | 1,024 GB | **8** quarter-A30 virtual | 6 GB | |
| **vdi** | 8 hours | 2 | 1 | 64 | 384 | **-** | | Used by GUI-interactive workload |

* **dev** partition shares nodes with the **normal** partition.

# 8 nodes/job are permitted to users whose code demonstrates efficient parallel performance scaling.

Job based limits

- Max nodes per job is 4 or 8#

User based limits

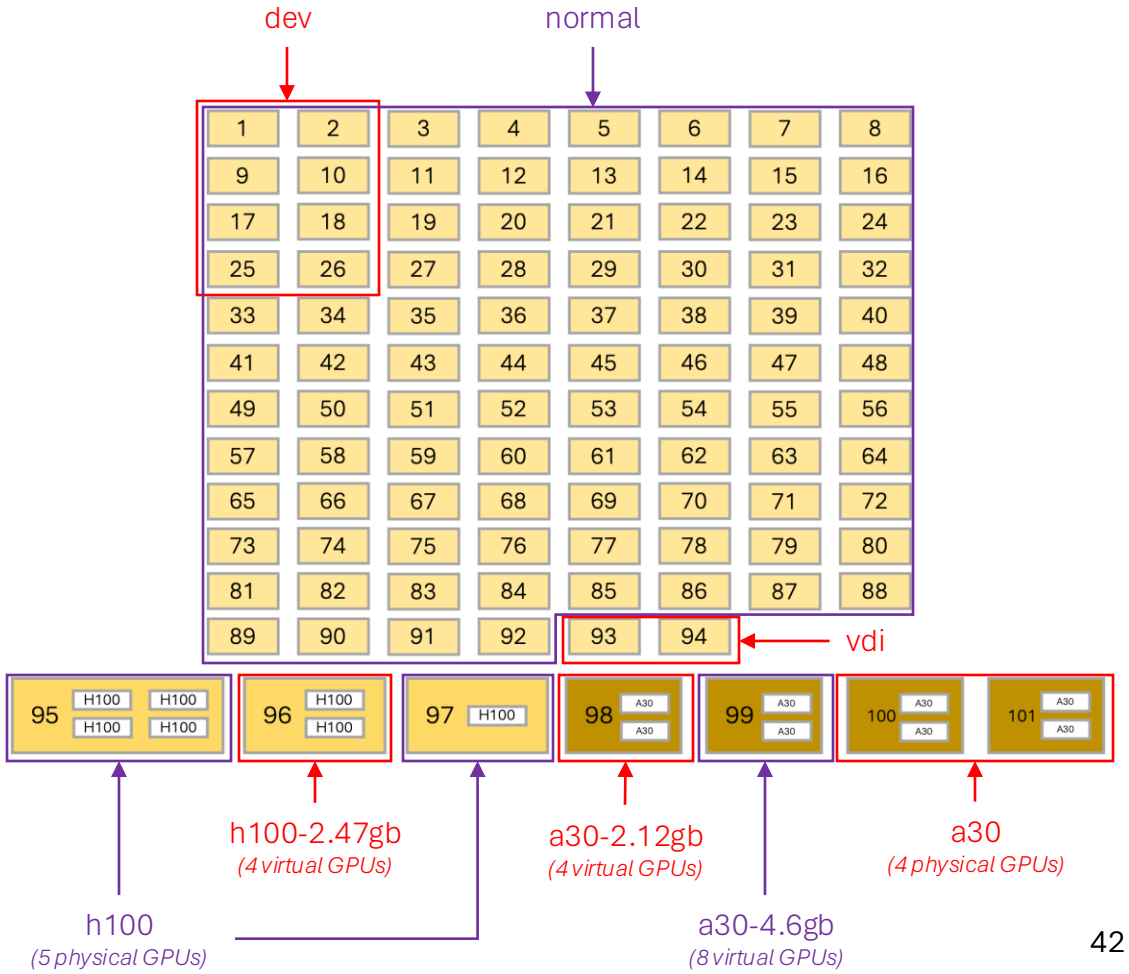
- Max running jobs per user is 4
- Max submitted jobs per user is 8

These limits can be relaxed for specific projects for limited time for codes that show efficient speedup at higher number of cores. Please contact us to temporarily raise your limit. We wish to support very large jobs on Juno but need to ensure that the jobs are using resources efficiently.

# Choosing the right partition for your job

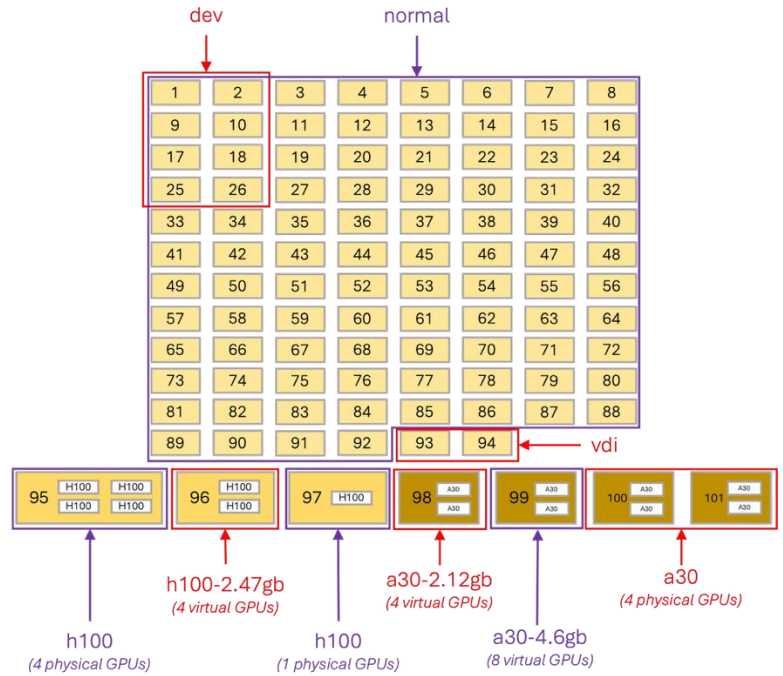
- Determine your job's resource needs?
  - How many nodes?
  - How many cores (CPUs) per node?
  - How much memory per node?
  - Do you need GPUs? If so, which kind?
  - How long does your program need to run?
- If your job needs 1 node, 6 CPU cores per node, 30GB memory and 3.5 hours to run, then your batch job file will have the following options specified:

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=6
#SBATCH --mem=30G
#SBATCH --time=3:30:00
#SBATCH --gres=gpu:1
#SBATCH --partition=h100
```

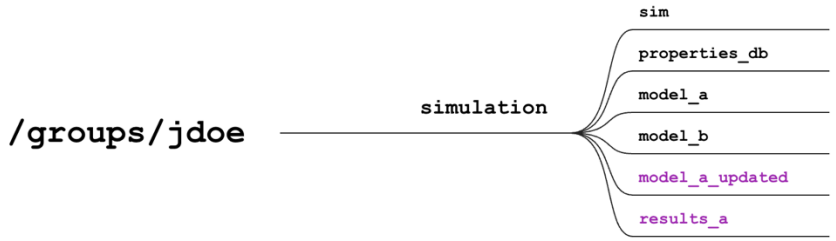


# Choosing the right partition examples

- Machine learning / Deep learning training with multiple GPUs
  - Light model: **`a30`** partitions
  - Big model: **`h100`** partitions
- OpenMP programs
  - **`normal`** partition, **`a30`** for more intensive RAM usage
- MPI programs
  - **`normal`** partition (no GPU needed), **`a30`** (GPU needed)
- Compiling, debugging, profiling
  - **`dev`** partition



# Scratch space




## Scratch

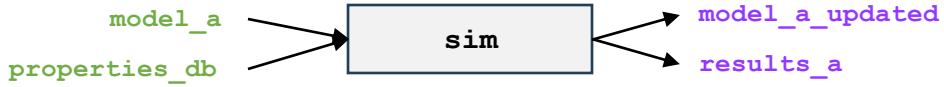
**~/scratch**

30TB (soft limit)

Private to user, never backed up, purged regularly




Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**




Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving

```
1  $ cd ~/scratch
   $ mkdir batch_job_a
   $ cd batch_job_a

   $ cp /groups/jdoe/simulation/sim .
   $ cp /groups/jdoe/simulation/properties_db .
   $ cp /groups/jdoe/simulation/model_a .

2  $ ./sim model_a results_a

3  $ cp model_a_updated /groups/jdoe/simulation
   $ cp results_a /groups/jdoe/simulation

4  $ cd ..
   $ rm -rf batch_job_a
```

# Scratch space

## Scratch

**~/scratch**

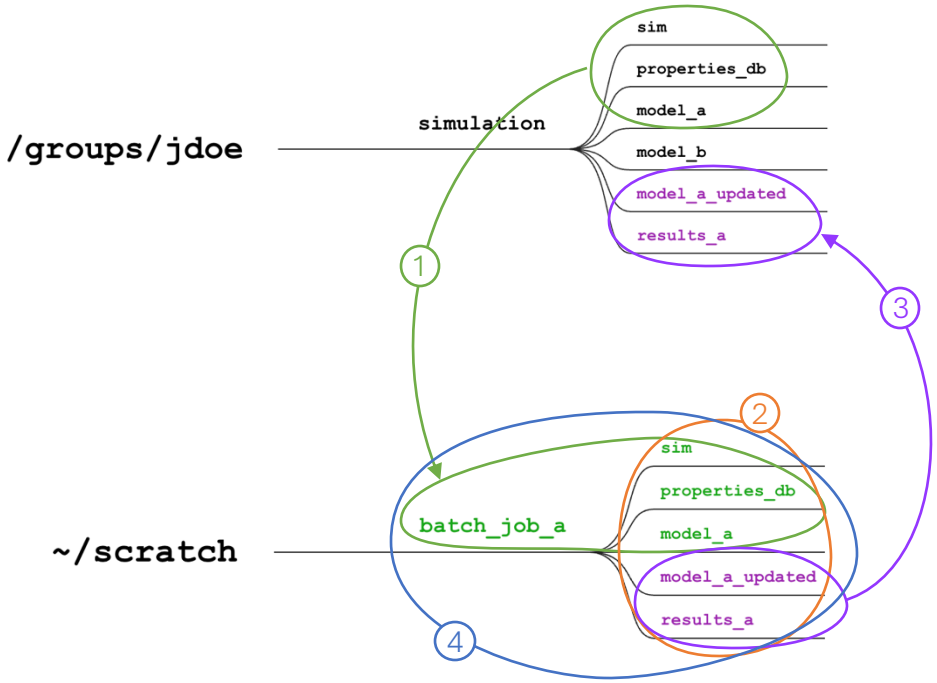
30TB (soft limit)

Private to user, never backed up, purged regularly

Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving




1
```
$ cd ~/scratch
$ mkdir batch_job_a
$ cd batch_job_a

$cp /groups/jdoe/simulation/sim .
$cp /groups/jdoe/simulation/properties_db .
$cp /groups/jdoe/simulation/model_a .
```

2
```
$ ./sim model_a results_a
```

3
```
$ cp model_a_updated /groups/jdoe/simulation
$ cp results_a /groups/jdoe/simulation
```

4
```
$ cd ..
$ rm -rf batch_job_a
```



# What's inside a SLURM job.sh?

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=6
#SBATCH --mem=30G
#SBATCH --time=3:30:00
#SBATCH --gres=gpu:1
#SBATCH --partition=h100
#SBATCH --job-name=sim_model_a
#SBATCH --output=sim_model_a_output.txt

# List and load the required modules
module list
```

```
# Batch files reside in the following directory
SRC=/groups/hpcre/orientation/simulation/

# Copy data IN at the start
cd ~/scratch
mkdir batch_job_a
cd batch_job_a

cp $SRC/sim .
cp $SRC/properties_db .
cp $SRC/model_a .

# Read from and write to Scratch during the job
# Run your job's commands
./sim model_a results_a

# Copy data OUT at the end
cp model_a_updated $SRC
cp results_a $SRC

# Clean up before leaving
cd ..
rm -rf batch_job_a
```

# Submitting and managing SLURM jobs

- **sbatch <batchfile>**
  - Submit a SLURM batch job, requires a batch job script filename
- **squeue --me**
  - List the queued and running batch jobs, use –u option if you only want to see your jobs
- **scancel <jobid>**
  - Cancel a submitted job, requires one or more job IDs

```
(base) [ksw240000@juno-l-02 simulation]$ pwd
/groups/hpcre/orientation/simulation
(base) [ksw240000@juno-l-02 simulation]$ ls
job.sh  model_a  model_b  properties_db  sim
(base) [ksw240000@juno-l-02 simulation]$ sbatch job.sh
Submitted batch job 2540797
(base) [ksw240000@juno-l-02 simulation]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           2540797 h100      sim_mode ksw24000  R       0:04      1 g-01-04
(base) [ksw240000@juno-l-02 simulation]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           2540797 h100      sim_mode ksw24000  R       0:17      1 g-01-04
(base) [ksw240000@juno-l-02 simulation]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
(base) [ksw240000@juno-l-02 simulation]$ ls
job.sh  model_a  model_a_updated  model_b  properties_db  results_a  sim  sim_model_a_output.txt
```



# Job output file

```
(base) [ksw240000@juno-l-02 simulation]$ cat sim_model_a_output.txt

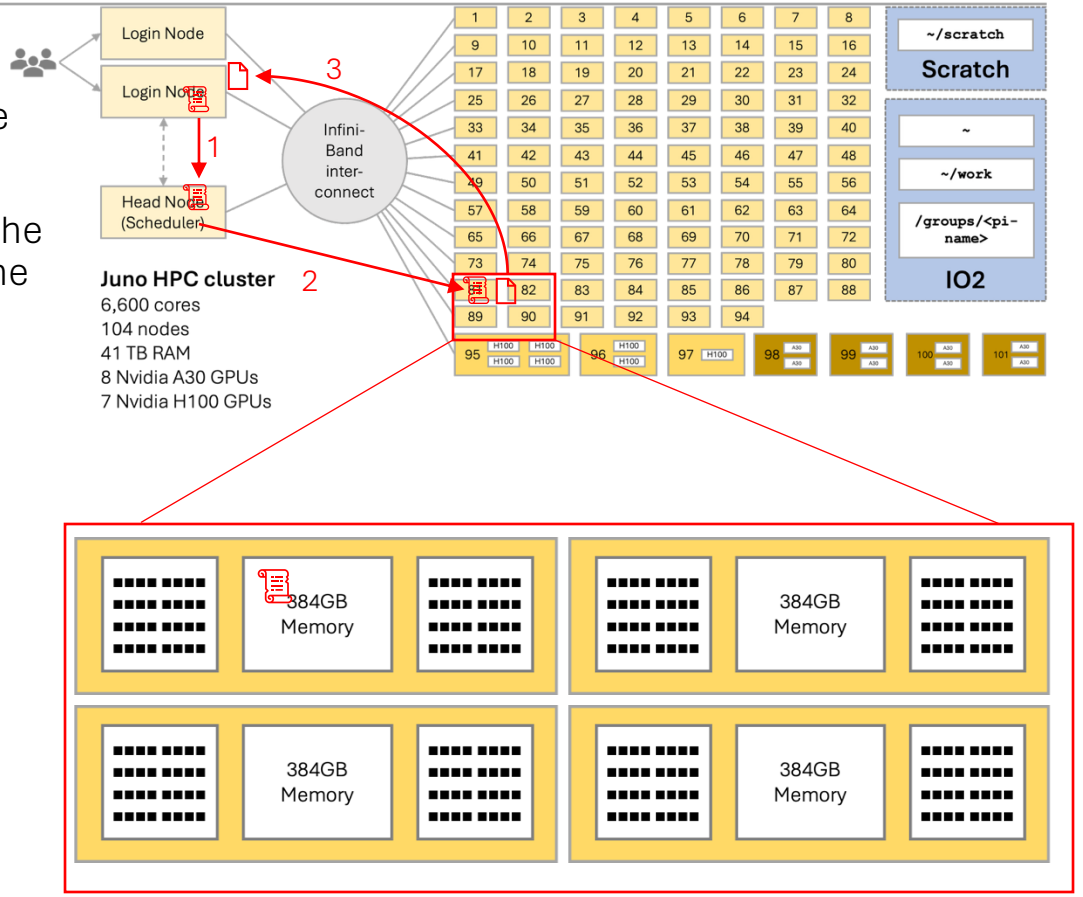
Currently Loaded Modules:
  1) autotools   3) gnu12/12.4.0   5) ucx/1.15.0          7) openmpi4/4.1.6
  2) prun/2.2    4) hwloc/2.7.2    6) libfabric/1.19.0    8) ohpc



Running simulation...
Input file: model_a
Output file: results_a
Reading model_a...
Reading properties_db...
Starting simulation
Writing results_a...
Saving restart file model_a_updated...
Completing simulation
```

# How does your batch job run?

1. **`sbatch job.sh`**
2. The scheduler allocates 4 nodes and starts the **`job.sh`** script on one of them
3. At end of the job, the output file is returned to the directory where the job was submitted (from the login node)



# Job scheduling algorithm

- To best serve our customers needs, we configure the scheduler optimize the following in a balanced way
  - Minimize the wait time of a user's jobs.
    - I.e., run the first job a user as soon as possible.
  - Maximize the number of users with running jobs.
    - I.e., run as many jobs from as many users concurrently as efficiently as possible.
  - Support large and long running computing jobs.
    - These represent the kind of computation that can only run on HPC systems.

The SLURM job scheduler computes the priority of each queued job and selects the job with highest priority to run next on the system.

The weight is calculated as follows.

Job's priority =

100,000 x (Fairshare of user) +
1,000 x (Job age factor) +
1,000 x (Job size factor)

- Each factor, its value and weight are show in the following table.

| Factor | Weight | Value of factor | Note |
|---|---|---|---|
| Fairshare | 100,000 | 0 to 1 | See below for full explanation |
| Age of job | 1,000 | 0 to 1 | 0 for new jobs and 1 for jobs 14 days or older |
| Size of job | 1,000 | 0 to 1 | Near 0 for small jobs, near 1 for large jobs (Size determined by CPUs, memory, time, and GPU resources of a job.) |

# Fairshare value

- In this example all numbers are selected for illustration (i.e., not real)
- All users start with share of 1.0
  - Using the system decreases share
  - Not using the system gradually increases share (over time)
  - Share fully restored in 2 weeks
  - Share reset to 1.0 at the start of the month

| Event | User A Share | User B Share |
|---|---|---|
| Initially | 1.0 | 1.0 |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

# How to monitor your share value?

- **`sshare`** tells you your, or every user's share value
- E.g.
- **`sshare`** (your share)
- **`sshare -a`** (all users' shares)

ksw240000@juno-l-01:~

```
juno(juno-l-01) ksw240000 ~ $  sshare -a
Account                    User  RawShares  NormShares    RawUsage  EffectvUsage  FairShare
-------------------- ---------- ---------- ----------- ----------- ------------- ----------
root                                          0.000000  2655590654      1.000000
 root                      root          1    0.333333           0      0.000000   1.000000
 hpcre                                   1    0.333333    26776917      0.010083
  hpcre                    amir          1    0.045455           0      0.000000   0.997297
  hpcre                 axm240026        1    0.045455        6382      0.000238   0.956757
  hpcre                 axv230014        1    0.045455        6507      0.000243   0.954054
  hpcre                 brd150330        1    0.045455          20      0.000001   0.962162
  hpcre                 cgd180001        1    0.045455          12      0.000000   0.964865
  hpcre                 dal676969        1    0.045455           0      0.000000   0.997297
  hpcre                 dal946607        1    0.045455      940376      0.035119   0.943243
  hpcre                 exa230016        1    0.045455           9      0.000000   0.967568
  hpcre                 kaa180001        1    0.045455    25054910      0.935691   0.940541
  hpcre                     kevin        1    0.045455           0      0.000000   0.997297
  hpcre                 ksw240000        1    0.045455      156986      0.005863   0.948649
  hpcre                 nxk230072        1    0.045455           0      0.000000   0.997297
  hpcre                 rgr071000        1    0.045455           0      0.000000   0.997297
  hpcre                      root        1    0.045455         859      0.000032   0.959459
```



# Other ways to run programs

# Ways to run your programs

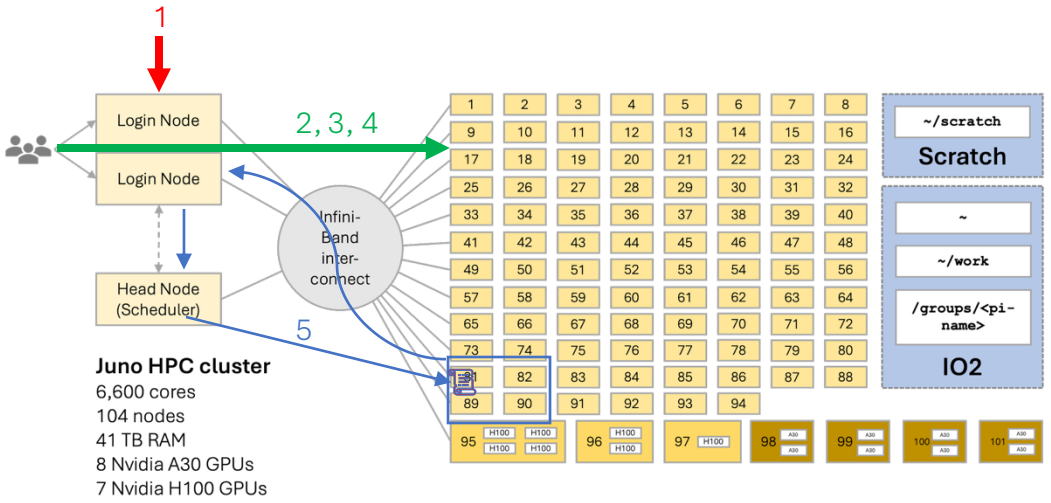
1. Interactively on login node from the command line

2. Interactively on compute nodes from **the command line** (next slide)

3. Interactively graphically using the **X Window Server**

4. Interactively graphically using **OOD** on compute nodes (already shown)

5. Batch processing on compute nodes (already covered)



# Running interactive session using SLURM

- Two steps
  1. Use **`salloc`** to acquire resources from SLURM
  2. Use **`srun`** to start your interactive session on the allocated resources/nodes

- **`salloc`** uses the same options as **`sbatch`** (nodes, tasks, time, GPU, etc.)

  **`$ salloc -p normal --mem=2GB`**

- **`srun`** is used to start a Bash shell interactively on the resource allocation

  **`$ srun --pty bash`**

  - Your prompt will change to show the node name (*e.g.* c-04-01)

```
(base) [dal281726@juno-l-01 ~]$ salloc -p normal --mem=2GB
salloc: Granted job allocation 69224
Disk quotas for user dal281726, inode (file) count and disk usage:
=========================  ====================  ================  ==========
Disk                       Usage                 Soft Limit        Hard Limit
=========================  ====================  ================  ==========
/home/dal281726            174k|42GB             300k|50GB         320k|55GB
/work/dal281726            16k|2.9GB             3.0M|1.0TB        3.1M|1.1TB
=========================  ====================  ================  ==========
(base) [dal281726@juno-l-01 ~]$ srun --pty bash
Disk quotas for user dal281726, inode (file) count and disk usage:
=========================  ====================  ================  ==========
Disk                       Usage                 Soft Limit        Hard Limit
=========================  ====================  ================  ==========
/home/dal281726            174k|42GB             300k|50GB         320k|55GB
/work/dal281726            16k|2.9GB             3.0M|1.0TB        3.1M|1.1TB
=========================  ====================  ================  ==========
(base) [dal281726@c-04-01 ~]$
```

# Running GUI programs using the X Window System

- MobaXterm and XQuartz offer free terminal with X11 server, tabbed SSH client, network tools and much more.

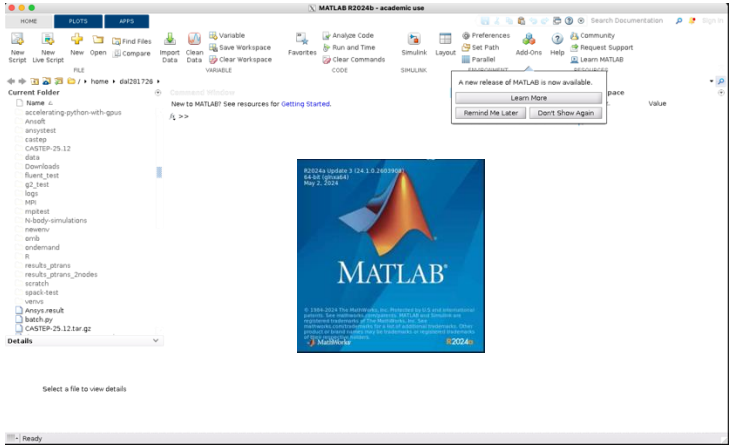
From MobaXterm (Windows):

```
$ ssh -X <your-NetID>@juno.utdallas.edu
$ xclock # launch a GUI-based clock
$ matlab
```




From XQuartz (Mac):

```
$ ssh -X <your-NetID>@juno.utdallas.edu
$ xclock # launch a GUI-based clock
$ matlab
```

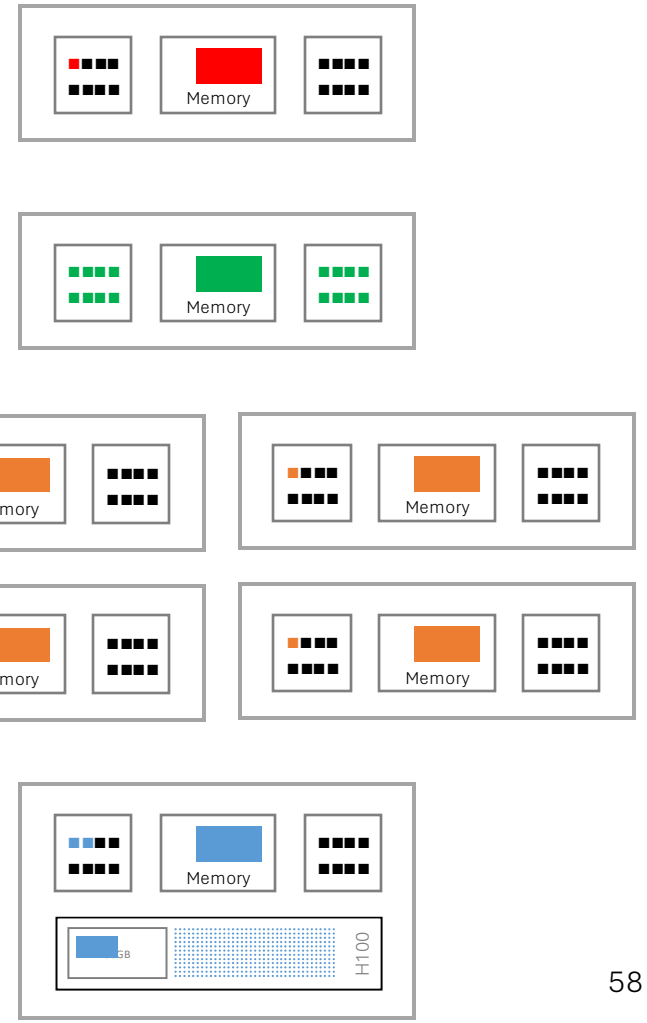




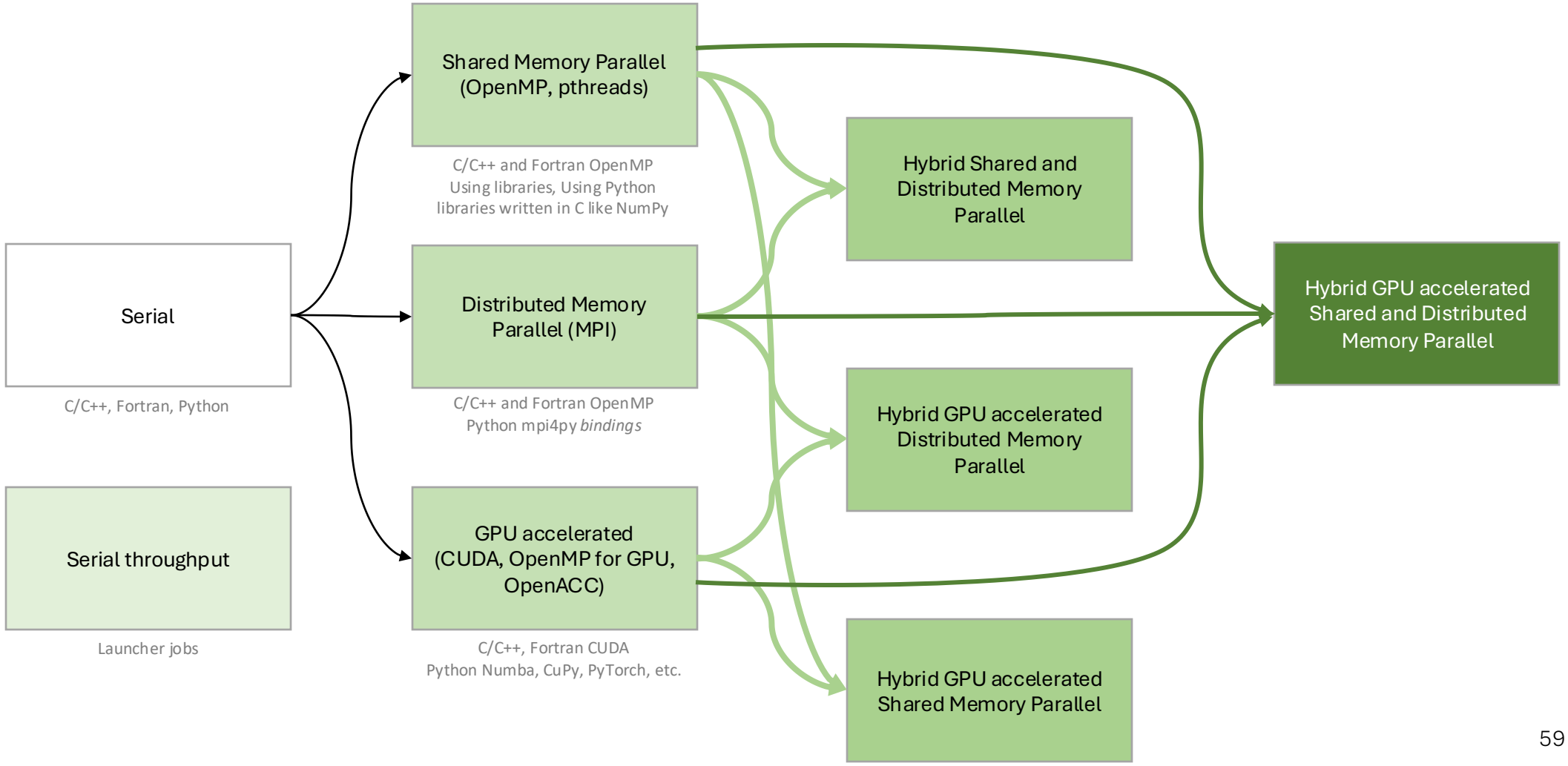
# Achieving high performance using parallel processing

# Achieving high performance on Juno

- Programs normally perform their work using just one core
  - This is called serial execution
- Achieving high performance requires the program to split its work and perform it simultaneously on multiple cores
  - This is called **parallel execution**
- HPC cluster support 3 modes of parallel execution
  - Shared memory parallel (aka, multithreaded parallel) mode uses multiple cores in one node
    - Program uses OpenMP for parallelization
  - Message passing parallel (aka, distributed memory parallel) uses multiple nodes
    - Program uses the MPI library for parallelization
  - GPU acceleration uses tiny CUDA cores for parallelism
    - Program uses CUDA or OpenMP for GPUs for GPU acceleration



# Models of parallel processing on HPC clusters



# Most common types of parallel programs

Serial



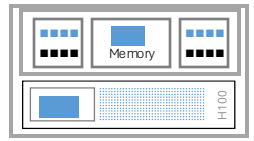

C/C++, Fortran, Python

Shared Memory Parallel (OpenMP, pthreads)




C/C++ and Fortran OpenMP
Using libraries, Using Python libraries written in C like NumPy

Hybrid GPU accelerated Shared Memory Parallel




Serial throughput



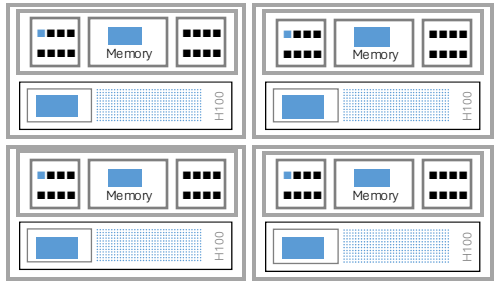
Launcher or SLURM array jobs

Distributed Memory Parallel (MPI)

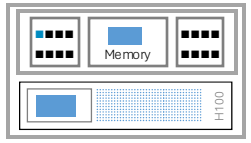



C/C++ and Fortran OpenMP
Python mpi4py *bindings*

Hybrid GPU accelerated Distributed Memory Parallel




GPU accelerated (CUDA, OpenMP for GPU, OpenACC)

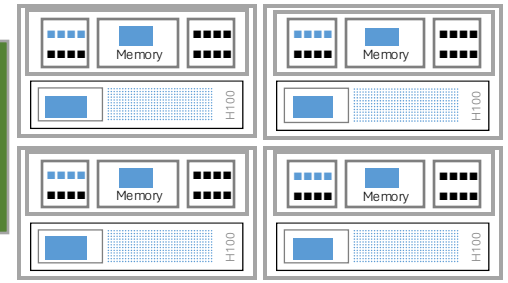



C/C++, Fortran CUDA
Python Numba, CuPy, PyTorch, etc.

Hybrid Shared and Distributed Memory Parallel




Hybrid GPU accelerated Shared and Distributed Memory Parallel





# How you program common types of parallel programs

| Serial | C/C++ | Fortran | Python | Containers |
|---|---|---|---|---|
| Serial | GNU compilers | GNU compilers | Python interpreter | Apptainer |
| Serial Throughput | Launcher | Launcher | Launcher | Launcher |
| Shared Memory Parallel | OpenMP, POSIX threads | OpenMP, POSIX threads | C/Fortran libraries (NumPy, BLAS, etc.) | N/A |
| Distributed Memory Parallel | MPI | MPI | mpi4py | MPI |
| GPU-accelerated | CUDA, OpenMP Offloading, OpenAcc | OpenAcc, OpenMP Offloading, CUDA Fortran | GPU libraries (CuPy, Pytorch, Tensorflow, etc.), Numba | *Depending on the language* |

**Upcoming workshops:** Accelerating Python with GPUs (October 2025), OpenMP and MPI programming



# Compiling and running your own code

a) OpenMP/multithreaded b) MPI c) GPU d) Python

# Serial programs

- Compile and run a serial C program

```
gcc -o crunch crunch.c
./crunch
```

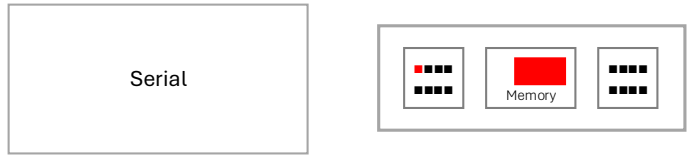
- **`crunch.c`** is the C language program or source file. Crunch is the compiled program file. After compilation, you can run **`crunch`** directly on the system.

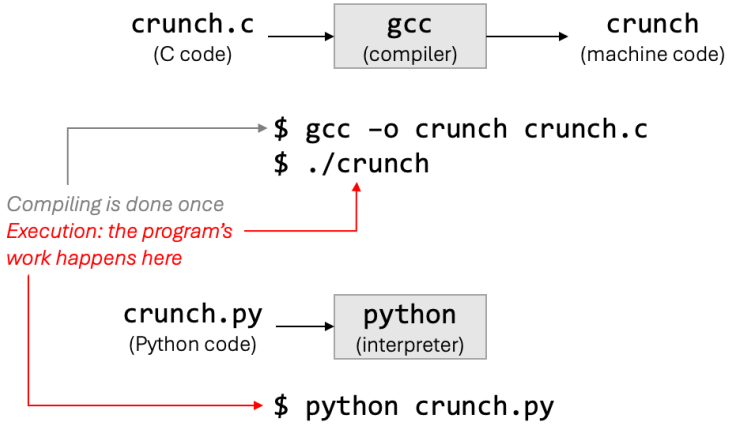
- Compile and run a serial Fortran program

```
gfortran -o crunch crunch.f90
./crunch
```

- Run a serial Python program

```
python crunch.py
```



# Shared memory parallel programs

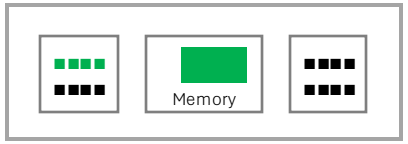
- Compile C program, run on 4 cores

```
gcc -fopenmp -o crunch_mt crunch_mt.c
export OMP_NUM_THREADS=4
./crunch_mt
```

- Compile Fortran program, run on 4 cores

```
gfortran -fopenmp -o crunch_mt crunch_mt.f90
export OMP_NUM_THREADS=4
./crunch_mt
```

Shared Memory Parallel
(OpenMP, pthreads)



# Message passing parallel programs

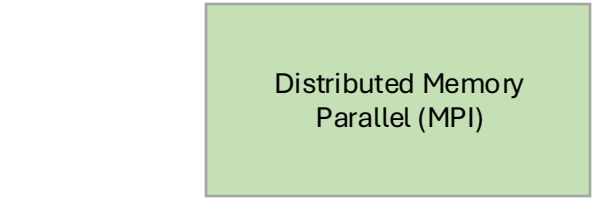
- Compile a distributed memory parallel C program and run it on 4 nodes

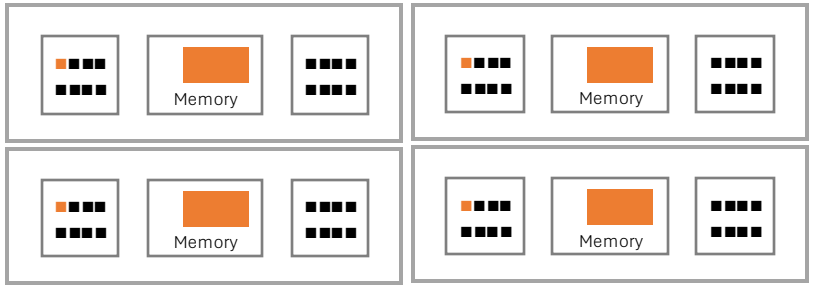
```
mpicc -o crunch_mpi crunch_mpi.c
mpirun –np 4 crunch_mpi
```

- Compile a distributed memory parallel Fortran program and run it on 4 nodes

```
mpif90 -o crunch_mpi crunch_mpi.f90
mpirun –np 4 crunch_mpi
```

Note: **`mpirun`** command must be run using SLURM batch system, it will not work on the login node. Why?



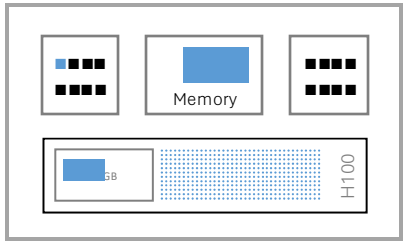



# GPU accelerated programs

- To compile and run a C CUDA program that runs on a GPU using the thousands of small processors on the GPU to accelerate computation, use the following commands

```
module unload gnu12
module load cuda
nvcc -o diffusion_cuda diffusion_cuda.cu
./diffusion_cuda
```

GPU accelerated
(CUDA, OpenMP for GPU, OpenACC)



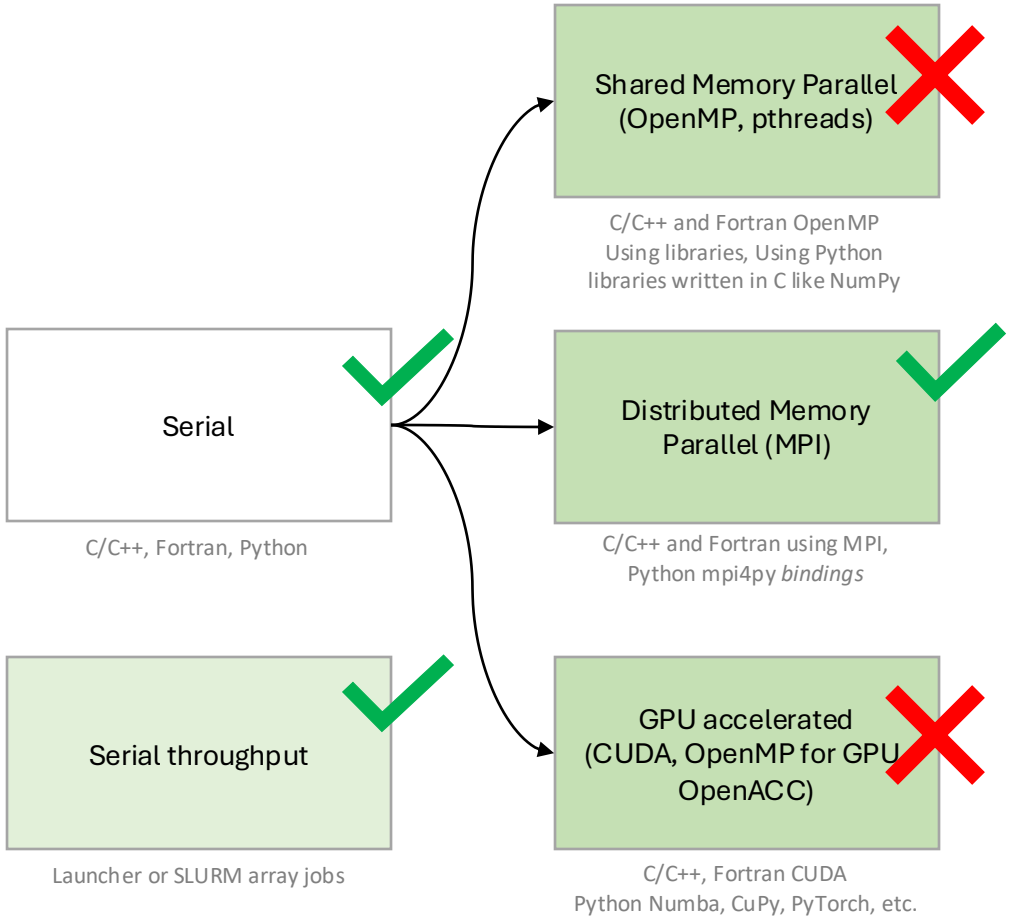

Note: The program must be run on a node with a GPU. If you run **`./diffusion_cuda`** on a node (e.g. login node) without a GPU, it may run but will produce incorrect results.



# Python programs using multiple CPUs and GPUs

- Concept of linking compiled code from other languages
- Linking modules/libraries written in OpenMP
- Linking modules/libraries written in CUDA
- Using **`miniconda`** to install software
  - Miniconda tutorial
- Why is Python slow?

# Can pure Python programs run in these serial and parallel modes?




Python's Global Interpreter Lock (GIL) prevents Python threads from running on multiple cores, only one thread runs on once core at a time.

Python programs can call modules utilizing library functions written in C. Those C functions can run in shared memory or multiheaded parallel mode. NumPy is an example.

The mpi4py module provides Python bindings for the MPI library.

GPU programs are written using C and CUDA (C extensions and library).

The two most common ways Python programs use GPUs are:

- Use Python modules with functions written in C and CUDA (e.g., CuPy, PyTorch)
- Use Numba which translates Python code into CUDA code

High-level Python APIs

├— **NumPy (CPU)**

├— **CuPy/PyTorch (GPU)**

↓

**MKL / BLAS (CPU) or cuDNN / cuBLAS (GPU)**

↓

CPU/GPU hardware



# Example of **miniconda** usage to install software (reference slide)

- Initialize and set up a **conda** environment

```
$ module load miniconda

$ conda create -n <env_name>
python=3.12 # This will be
installed in your /home directory

$ conda create -p <path_to_env>
python=3.12 # Recommended

$ conda init bash

$ source ~/.bashrc

$ conda activate <env_name>
```

- Installing software and packages

```
$ conda install scipy

$ conda install -c conda-forge
tensorflow

$ conda list

$ conda remove scipy

$ pip install --upgrade MDAnalysis
```

# Hybrid shared memory and message passing programs

- To compile a C hybrid shared and distributed memory parallel program and run it on 4 cores/node on 4 nodes (total 16 cores)

```
mpicc -fopenmp -o crunch_mpi_mt
crunch_mpi_mt.c

export OMP_NUM_THREADS=4

mpirun –np 4 crunch_mpi_mt
```

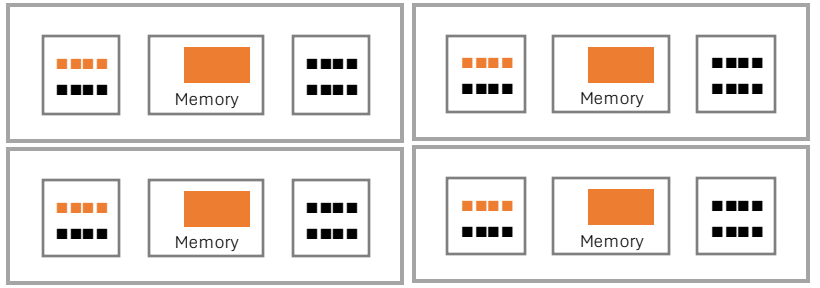
Hybrid Shared and Distributed Memory Parallel

- To compile a Fortran hybrid shared and distributed memory parallel program and run it on 4 cores/node on 4 nodes (total 16 cores)

```
mpif90 -fopenmp -o crunch_mpi_mt
crunch_mpi_mt.f90

export OMP_NUM_THREADS=4

mpirun –np 4 crunch_mpi_mt
```




Note: **mpirun** command must be run using SLURM job scheduler, it will not work on the login node.

# Hybrid shared memory and GPU accelerated

- To compile a C hybrid shared memory parallel and GPU-accelerated program and run it on 4 cores

```
module unload gnu14

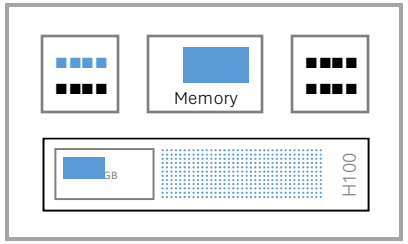
module load cuda

nvcc -Xcompiler -fopenmp
hybrid_omp_cuda.cu -o hybrid_omp_cuda

export OMP_NUM_THREADS=4

./hybrid_omp_cuda
```

Hybrid Shared Memory Parallel and GPU-accelerated



# Hybrid message passing and GPU accelerated

- To compile a C hybrid shared and distributed memory parallel and GPU-accelerated program and run it on 4 cores

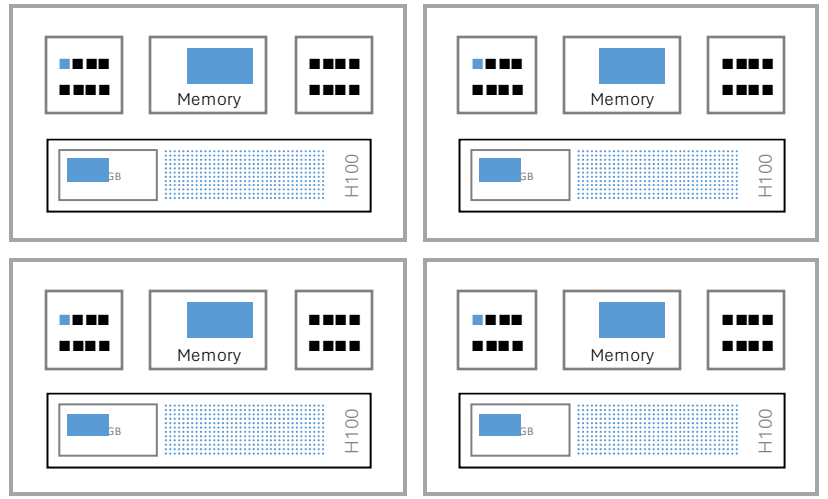
```
module unload gnu14

module load cuda

nvcc -Xcompiler -ccbin=mpicxx
hybrid_mpi_cuda.cpp -o hybrid_mpi_cuda

mpirun -n 4 ./hybrid_mpi_cuda
```

Hybrid Distributed Memory Parallel and GPU acceleration



# Hybrid “everything” programs

- To compile a C hybrid shared and distributed memory parallel and GPU-accelerated program and run it on 4 cores

```
module unload gnu14

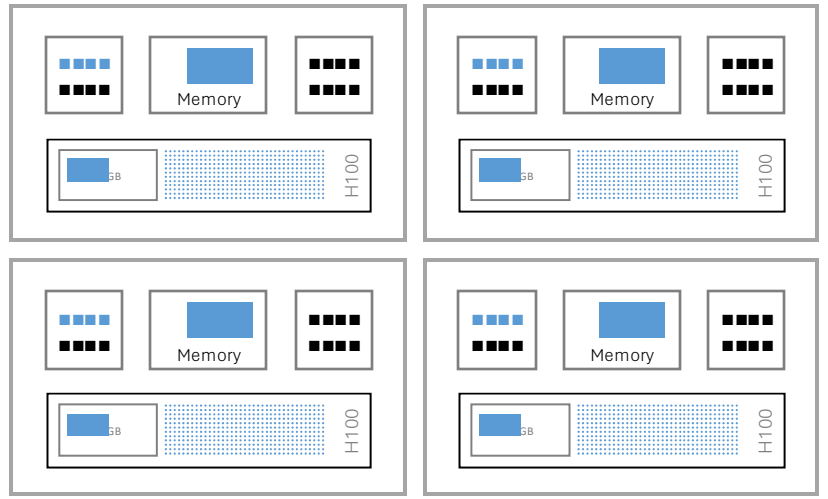
module load cuda

nvcc -Xcompiler -fopenmp -ccbin=mpicxx
hybrid_openmp_mpi_cuda.cpp -o
hybrid_openmp_mpi_cuda

export OMP_NUM_THREADS=4

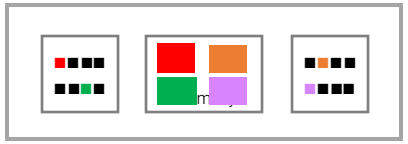
mpirun -n 4 ./hybrid_openmp_mpi_cuda
```

Hybrid Distributed Memory Parallel and GPU acceleration



# High throughput workloads using Launcher

High throughput



```
# tasklist.txt
python myscript.py input_1.txt output_1.txt
python myscript.py input_2.txt output_2.txt
python myscript.py input_3.txt output_3.txt
...
python myscript.py input_100.txt output_100.txt
```

```
# job.sh
#!/bin/bash
#SBATCH --job-name=py_launcher
#SBATCH --nodes=2
#SBATCH --partition=normal
#SBATCH --ntasks-per-node=64
#SBATCH --time=00:30:00
#SBATCH --output=py_launcher.out
#SBATCH --error=py_launcher.err

# Load required modules
module load launcher

# Define paths
export LAUNCHER_WORKDIR=$PWD
export LAUNCHER_JOB_FILE=$PWD/tasklist.txt

export LAUNCHER_PPN=64   # tasks per node
export LAUNCHER_NHOSTS=2
echo "Starting high-throughput Python tasks..."
$LAUNCHER_DIR/paramrun
echo "All tasks completed."
```

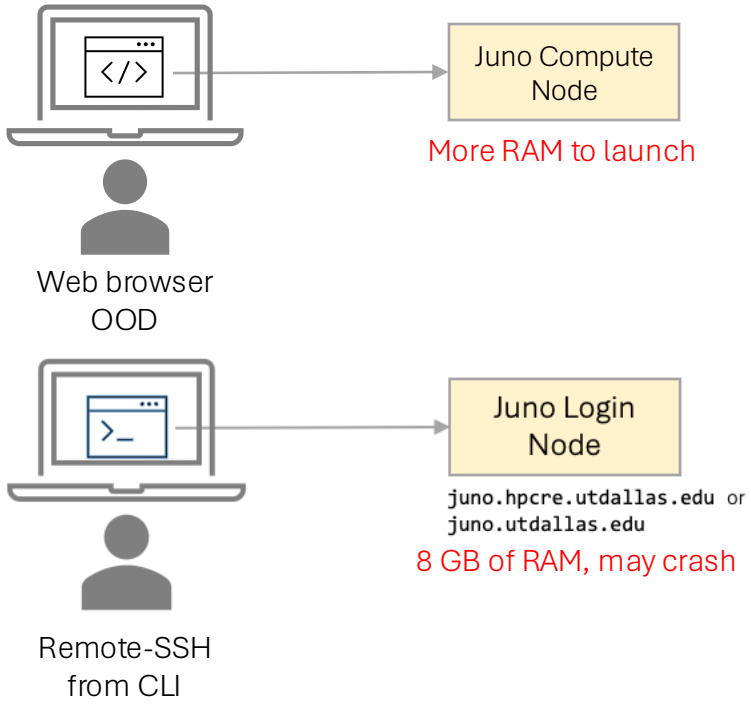
```
(base) [dal281726@juno-l-02 ~]$ pwd
/home/dal281726
(base) [dal281726@juno-l-02 ~]$ ls
input_1.txt input_2.txt input_3.txt input_4.txt input_5.txt input_6.txt input_7.txt
input_8.txt input_9.txt ... input_100.txt job.sh myscript.py tasklist.txt
(base) [dal281726@juno-l-02 ~]$ sbatch job.sh
Submitted batch job 2540797
```

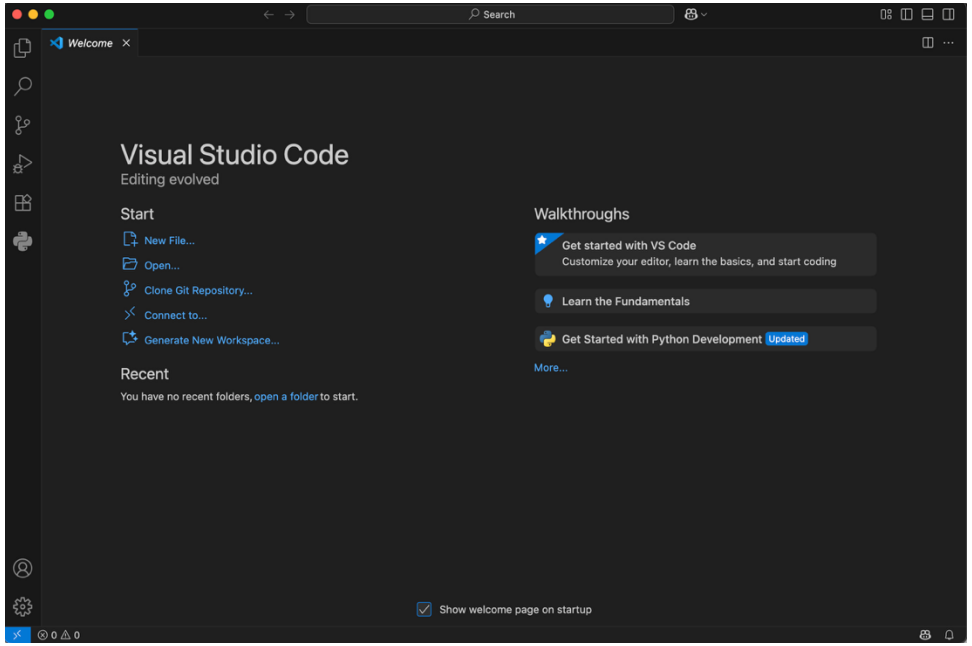
# Compilers available on Juno

- GCC (GNU compiler collection): versions 5, 8, 9, 11, 12, 13, 14
- Intel 2025.0
- Nvidia toolkits: cuda 11.7, 12.4, 12.6
- MPI toolchains: mpich, openmpi, prun

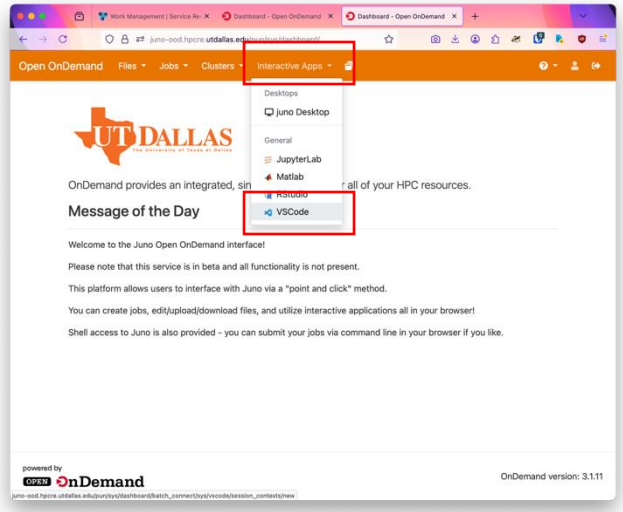
# IDE: VS Code

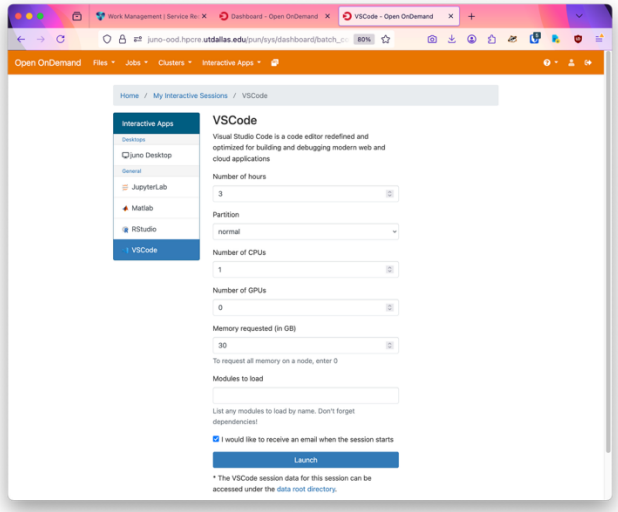
- Running on Open OnDemand
- Running on your workstation using Remote-SSH extension (not recommended)
  - A limit of 8GB of RAM on Juno login nodes

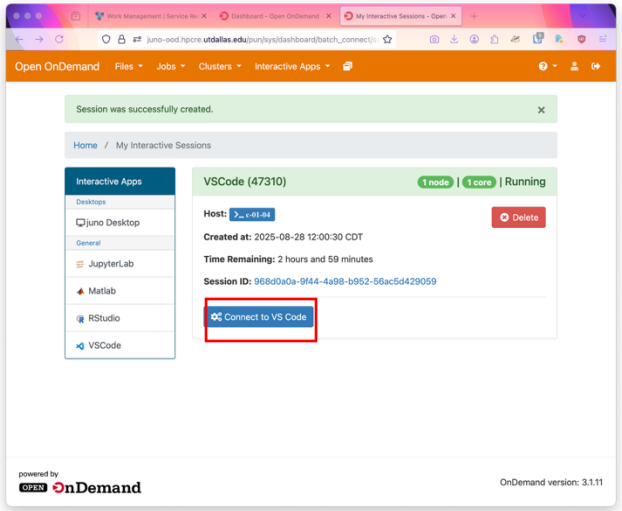


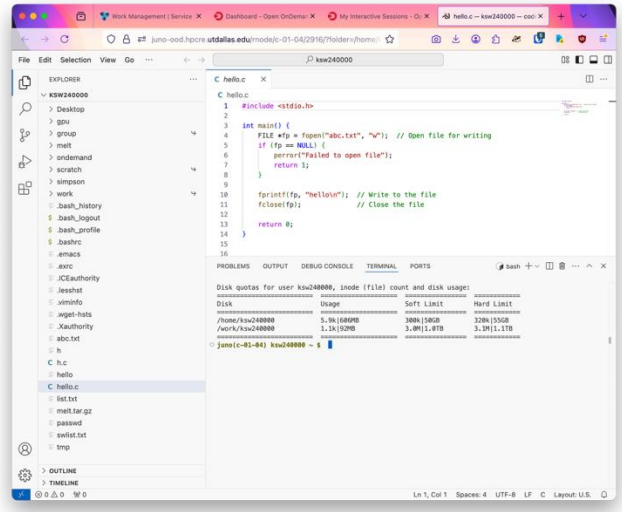



# VSCode on Open OnDemand





# Running pre-packaged programs

- Matlab (CLI, GUI)
- Gaussian
- Fluent (CLI, GUI)
- Containers



# MATLAB

- Command line interface

1. Running interactively: request a compute node and start an interactive session, then load and run the script

```
$ salloc -p normal --mem=2GB
$ srun --pty bash
$ module load matlab
$ matlab -nodisplay -nosplash -nodesktop -r
"run('path/to/your/script.m'); exit;"
```

2. Running as a batch job:

```
#slurm.sh
$ module load matlab --mem=2GB
$ matlab -nodisplay -nosplash -nodesktop -r
"run('path/to/your/script.m'); exit;"
```

# MATLAB

- With a graphical user interface (MobaXterm, XQuartz):

Make sure that you logged in to Juno with the -X option:

```
$ ssh -X <netID>@juno.utdallas.edu
```

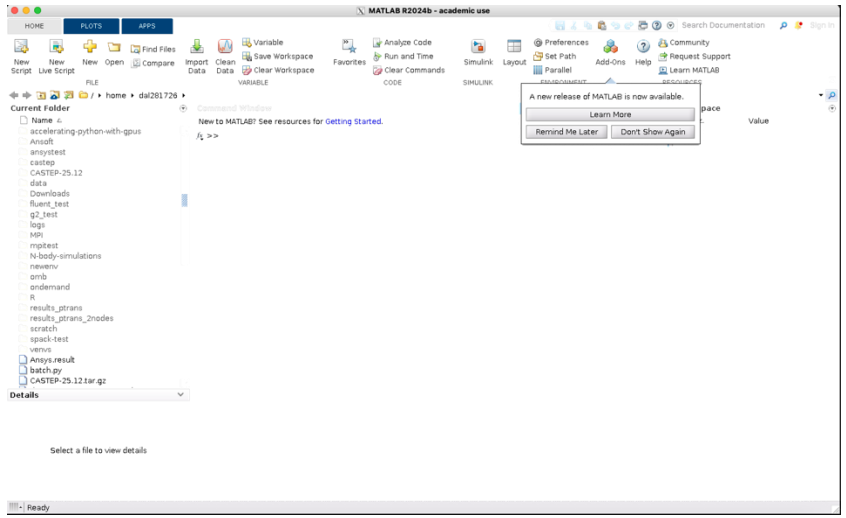
Request a compute node and start an interactive session

```
$ salloc -p normal --mem=2GB
$ squeue --me #to check which node is assigned to you
$ ssh -X c-04-01
```

Load the module and run

```
$ module load matlab
$ matlab
```

```
(base) [dal281726@juno-l-01 ~]$ salloc -p normal --mem=2GB
salloc: Granted job allocation 69229
Disk quotas for user dal281726, inode (file) count and disk usage:
========================  ====================  ================  ============
Disk                      Usage                 Soft Limit        Hard Limit
========================  ====================  ================  ============
/home/dal281726           174k|42GB             300k|50GB         320k|55GB
/work/dal281726           16k|2.9GB             3.0M|1.0TB        3.1M|1.1TB
========================  ====================  ================  ============
(base) [dal281726@juno-l-01 ~]$ squeue --me
             JOBID    PARTITION                NAME      USER ST       TIME  NODES NODELIST(REASON)
             69082 new-cpu-test                Abu4 dal281726  R   18:09:59      1 c-06-01
             69229       normal         interactive dal281726  R       0:04      1 c-04-01
             69224       normal         interactive dal281726  R      17:29      1 c-04-01
(base) [dal281726@juno-l-01 ~]$ ssh -X c-04-01
Warning: Permanently added 'c-04-01' (ED25519) to the list of known hosts.
Disk quotas for user dal281726, inode (file) count and disk usage:
========================  ====================  ================  ============
Disk                      Usage                 Soft Limit        Hard Limit
========================  ====================  ================  ============
/home/dal281726           174k|42GB             300k|50GB         320k|55GB
/work/dal281726           16k|2.9GB             3.0M|1.0TB        3.1M|1.1TB
========================  ====================  ================  ============
(base) [dal281726@c-04-01 ~]$ module load matlab
(base) [dal281726@c-04-01 ~]$ matlab
MATLAB is selecting SOFTWARE rendering.
```



# Gaussian

- Command line interface

1. Running interactively: request a compute node and start an interactive session, then load and run the script

```
$ salloc -p normal --mem=2G
$ srun --pty bash
$ module load gaussian
$ g16 input.com > output.log
```

2. Running as a batch job:

```
#slurm.sh
$ module load gaussian
$ g16 input.com > output.log
```

Gaussian automatically uses **`~/scratch`** to store **`.rwf`** files during batch jobs, you don't need to specify the path for these files.

# Ansys Fluent

- Command line interface

1. Running interactively: request a compute node and start an interactive session, then load and run the script

```
$ salloc -p normal --mem=2GB
$ srun --pty bash
$ module load ansys/2025R1
$ fluent 2d -g -i sample.jou
```

2. Running as a batch job:

```
#slurm.sh
$ module load ansys/2025R1
$ fluent 2d -g -i sample.jou
```

# Ansys Fluent

- With a graphical user interface (MobaXterm, XQuartz):

Make sure that you logged in to Juno with the -X option:

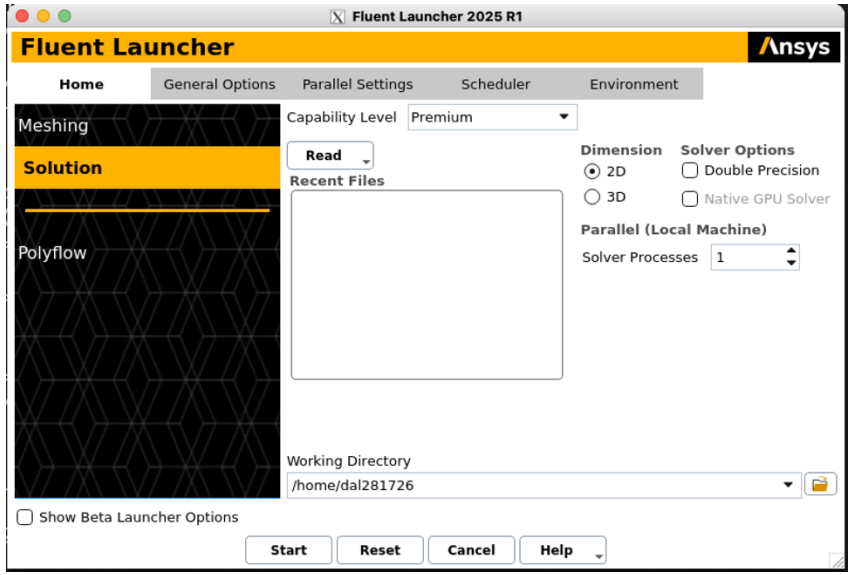
```
$ ssh -X <netID>@juno.utdallas.edu
```

Request a compute node and start an interactive session

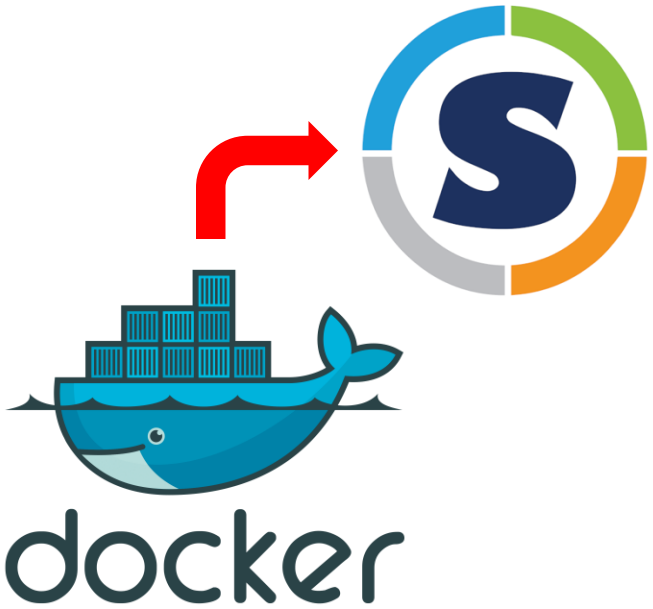
```
$ salloc -p normal --mem=2GB
$ squeue --me #to check which node is assigned to you
$ ssh -X c-04-01
```

Load the module and run

```
$ module load ansys/2025R1
$ fluent
```



# Containers




- Docker normally runs as the root user, which gives it full control of the system.
- On shared HPC clusters, that's a big security risk, because one user's container could potentially affect others or the system itself.
- You can convert your Docker images to Singularity/Apptainer images and run on Juno

```
#!/bin/bash
#SBATCH --job-name=apptainer_test
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --time=00:10:00
#SBATCH --output=apptainer_test.out
#SBATCH --error=apptainer_test.err

module load apptainer
# Define paths
CONTAINER=/path/to/my_container.sif
WORKDIR=$PWD

echo "Running Python script inside Apptainer container..."
apptainer exec \
    --bind $WORKDIR:/work \
    $CONTAINER \
    python /work/myscript.py arg1 arg2
echo "Done!"
```



# Getting help

Juno is an HPC cluster

Basic Unix commands summary

Directories and data management

Running programs on Juno

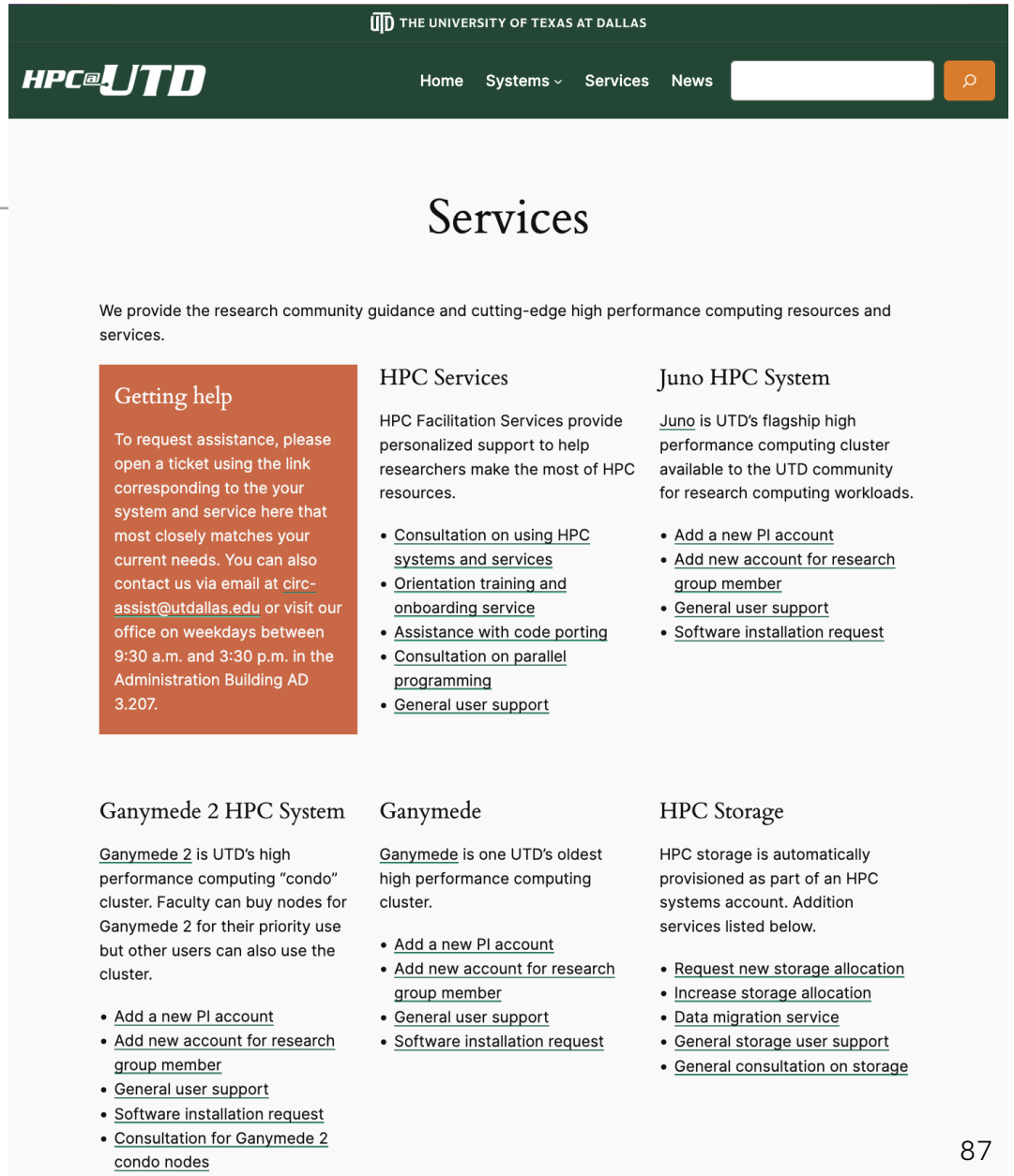
Introduction to the SLURM job scheduler

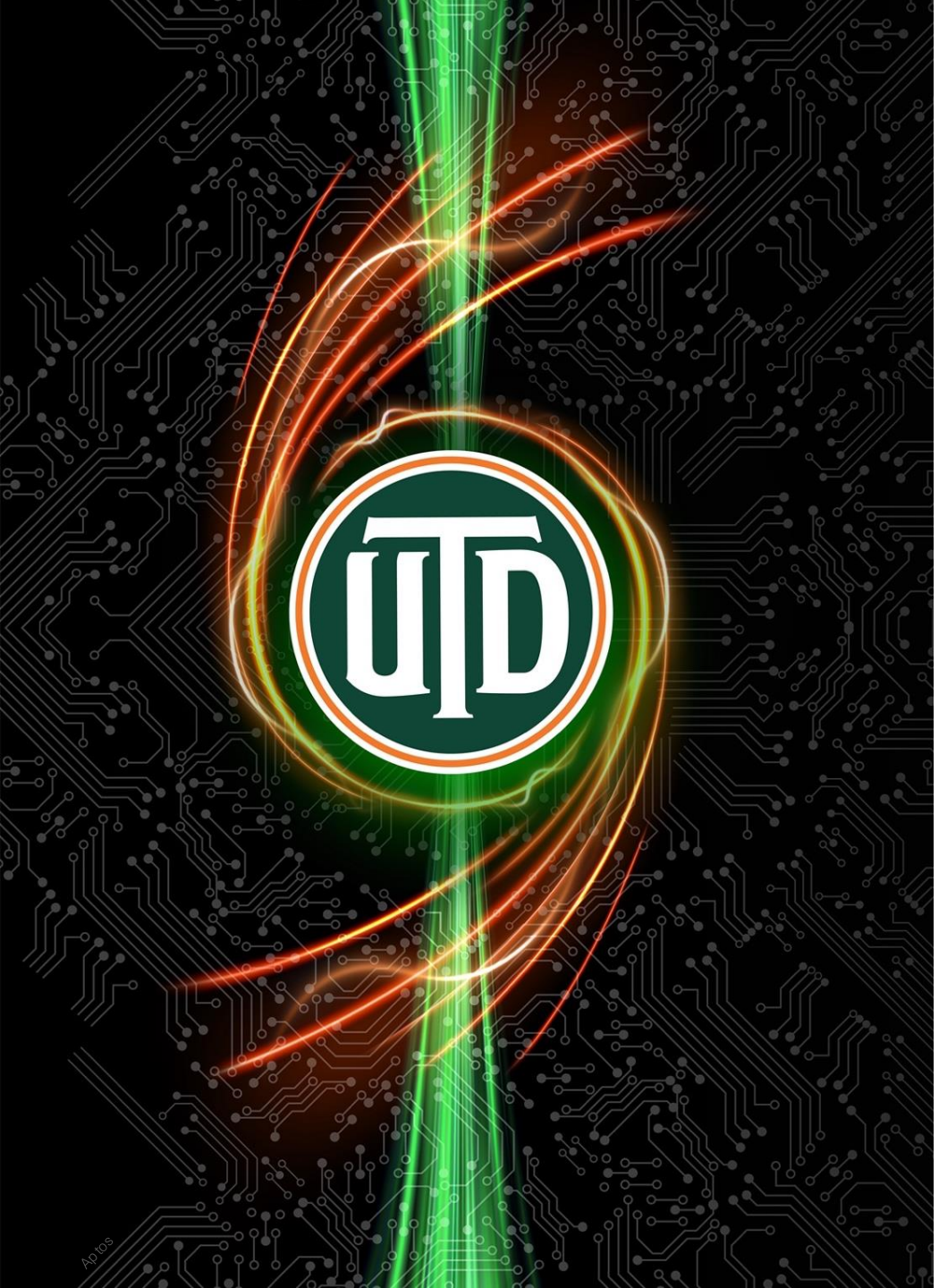
Getting help

# Getting help

1. Please visit the HPC website's services page https://hpc.utdallas.edu/services/
2. Locate your area/system
   - HPC services
   - Juno
   - Ganymede2
   - Ganymede
   - HPC storage
3. Click on the link that most closely matches your needs

- When unsure, please email circ-assist@utdallas.edu

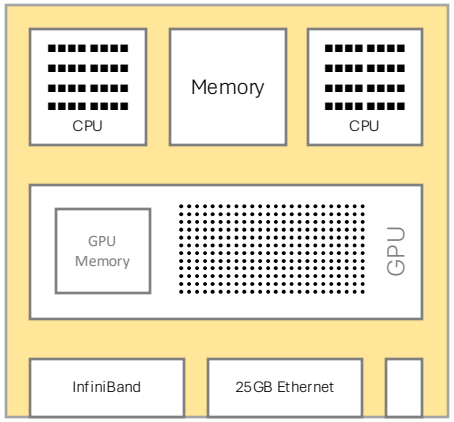


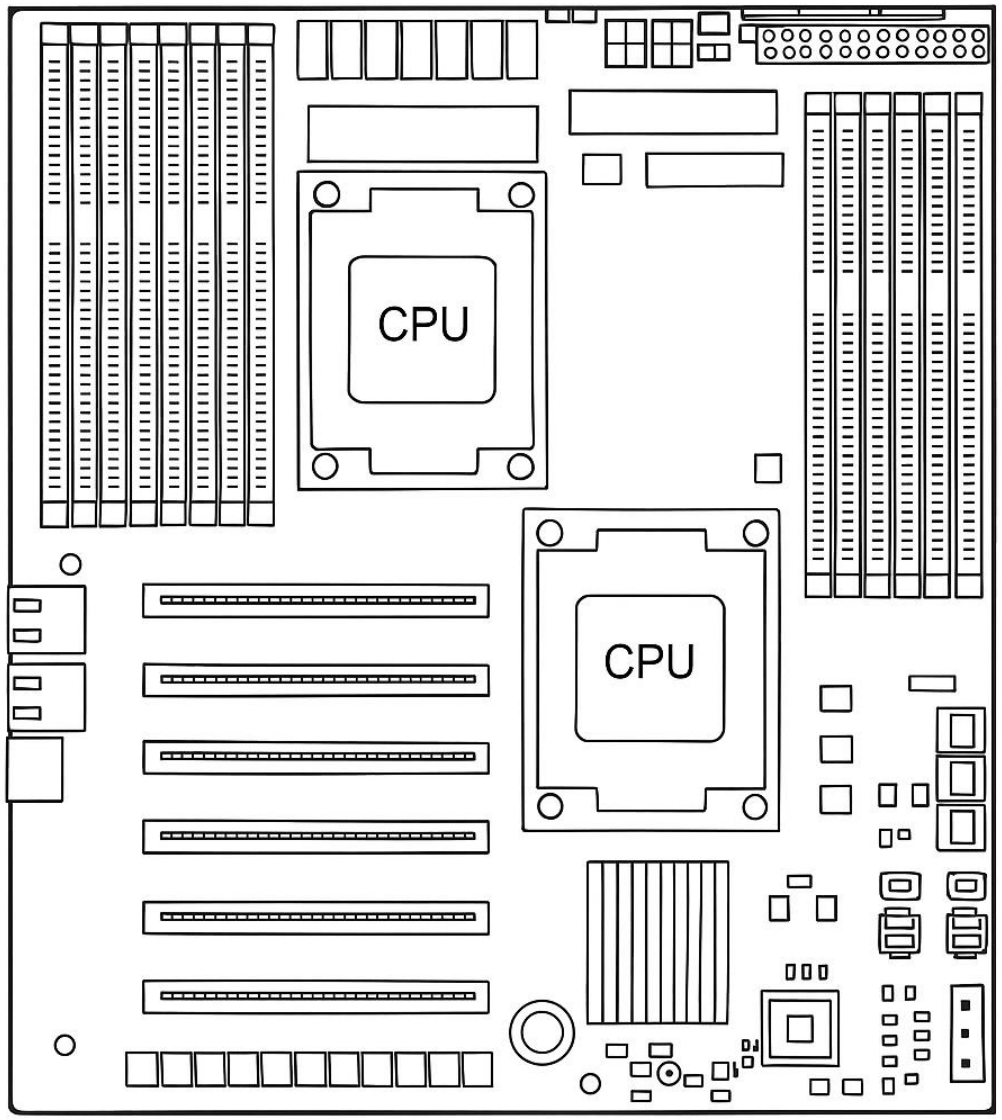



# Thank you

Questions? Comments? Feedback?

HPC@UTD


CPU
Memory
CPU
GPU Memory
GPU
InfiniBand
25GB Ethernet


CPU
CPU